

Imperial College London

ELEC50008 2ND YEAR PROJECT

IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC
ENGINEERING

Minotaur's Thread

Authors:

Chang Liu CID: 02108888,

Chenglin Sun CID: 02030498,

George Melissinos CID: 02066339,

Anastasis Varvarigos CID: 02053685,

Mengjie Zhang CID: 02041373,

Qidong Zhou CID: 02042327

GitHub Repository:

https://github.com/ChangLiu2022/SecondYearFinal_Team26

Word Count: **9928**

June 20, 2023

Contents

1	Introduction	4
1.1	Abstract	4
1.2	System overview	4
1.3	Project management	6
1.3.1	Job allocation	6
2	Mechanical	9
2.1	Overview	9
2.2	Manufacturing Constraint	10
2.3	Iteration History	12
3	Control System	13
3.1	Sensing	13
3.1.1	Acceleration, velocity and distance	13
3.1.2	Pitch angle	16
3.2	Balancing	19
3.2.1	Design concept	19
3.2.2	Modeling and simulation	22
3.2.3	Real time sensor measurement	23
3.2.4	Controller design	23
3.3	Linear motion	27
3.3.1	Design concept	27
	Duo wheel system	27
3.3.2	Modeling and simulation	27
	Physics	27
3.3.3	Dead reckoning	28
3.4	z-x plane turning	29
3.4.1	Design concept	29
3.4.2	Modeling and simulation	29
3.4.3	Dead reckoning	30
3.4.4	Turn angle controller	30
4	Power System	37
4.1	Problem Definition	37
4.2	Top-level functional design	37
4.3	Implementation and Design choices	38

4.4	Future development	42
5	Camera vision	44
5.1	HSV vs RGB	44
5.1.1	HSV Advantages	44
5.1.2	HSV Limitations	45
5.2	Filtering	45
5.2.1	1-Dimensional Filter	45
5.2.2	2-Dimensional Filter	46
5.3	Method of Detection	46
5.3.1	For White Lines	47
5.3.2	For Beacons	48
5.4	Transmitting with ESP32	48
6	Communication	50
6.1	Communication between FPGA and ESP32	50
6.2	Communication between ESP32 and Server	51
6.3	Communication between Control and Vision ESP32s	51
6.4	Conclusion	52
7	Data processing	53
7.1	Initial Setup	53
7.2	Database Design	53
7.3	Backend Overview	54
7.4	Server Configuration	54
7.5	Database Connection	54
7.6	Data Manipulation	55
7.7	Frontend Overview	55
7.8	Application Layout	55
7.9	VUE Components	56
7.10	Maze Frame Details	57
7.10.1	Template Section	57
7.10.2	Script Section	57
7.10.3	Style Section	58
7.11	Older Version	58
8	Navigation / Backend Algorithm	60
8.1	Positioning	60
8.1.1	Overview	60
8.1.2	Code Implementation	61
8.2	Mapping	63
8.2.1	Transformation	63
8.2.2	Mapping Routine	64
8.2.3	Mapping Post-processing	64
8.3	Decision Making	65
8.3.1	Overview	65

8.3.2	Second Iteration	65
8.3.3	First Iteration	67
9	Conclusions	68
10	References	69
11	Appendix	70
11.1	Robot specification	70
11.2	List of hardware components	72
11.3	Deduction of Positioning Algorithm	73

Chapter 1

Introduction

1.1 Abstract

The group name, "The Minotaur's Thread", inspired by Greek Mythology, symbolizes the purpose of the Rover, which is to move around the labyrinth and find its way out, despite the difficulties that arise represented by the "Minotaur". The "thread" is the optimal solution to find its way back, in Rover's case the Algorithm.

The conduction of this cause is performed by dividing the problem into sub-modules, according to the requirements of the given tasks, and then integrating them into a functional system. The five subsystems as seen in Figure 1.1: Control, Energy, Vision, Command, Navigation.

The Control subsystem is responsible for balance, which involves control of the robot's pitch angle while granting it robust stability and movement, which involves control of the precise location and position of the robot while keeping it balanced vertically.

The Energy subsystem controls a solar configuration to ensure correct energy distribution to three illuminated beacons.

For the Vision subsystem, the camera detects positions of beacons and patterns of lines which then can be used to calculate the position of the rover and how it should move.

The Command subsystem is responsible for data processing, including storing, querying and displaying of the data.

Navigation subsystem, also known as backend algorithm, uses the data transmitted and conduct the positioning, mapping, and decision making for the robot.

1.2 System overview

Structural Design

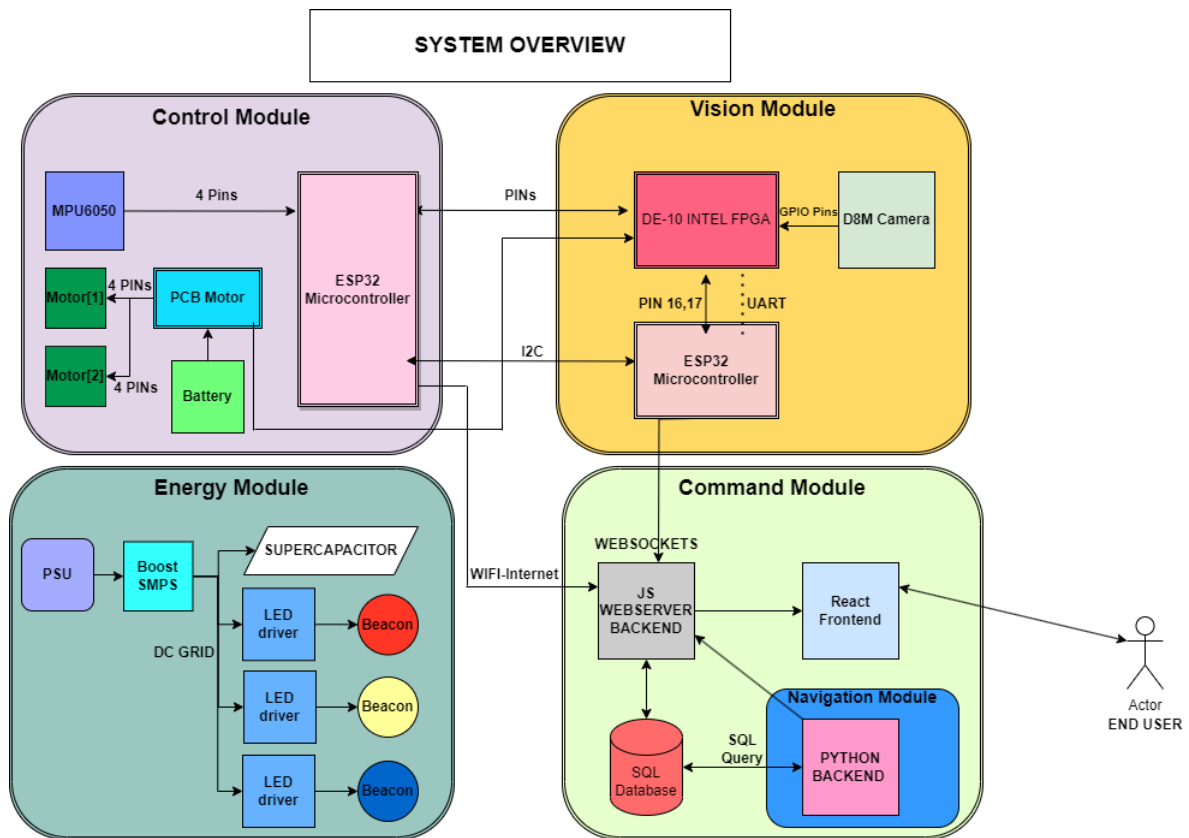


Figure 1.1: System overview

1.3 Project management

1.3.1 Job allocation

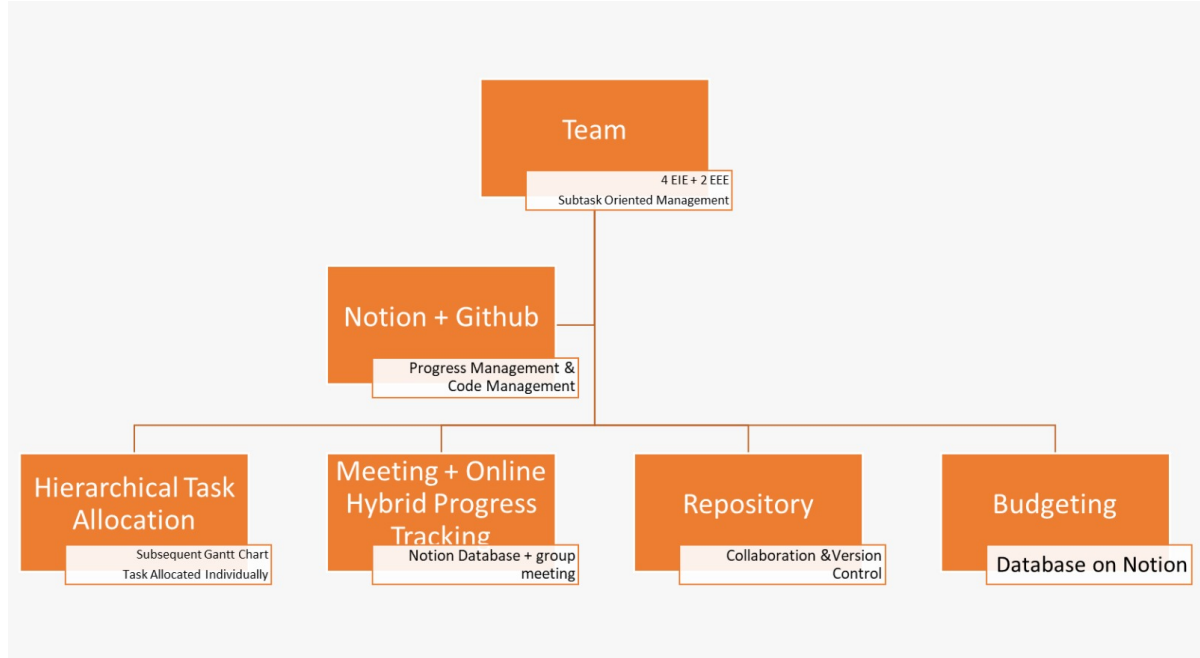


Figure 1.2: Project management

Our team consists of 4 EIE students and 2 EEE students each in charge of areas of their own specialism.

Chang was in charge of backend algorithm for navigation and mapping the maze; Mengjie worked on designing the control system; George was in charge of power system and beacon control; Anastasis worked on communications using the ESP32s and on the integration of the subsystems; Chenglin and Qidong worked on the computer Vision.

Teamworking and inter-communication was carried out daily for efficiently exchanging ideas, thereby sustaining a successful engineering design process.

Aa Name	Urgency	Major Tasks	Deadline	Status	Person
Basic OpenCV Mapping	Mid	Image Algorithm	May 30, 2023	Done	Chang
Positional control model	Mid	Control	May 23, 2023	Done	Mengjie Zhang
Turn angle control model	Mid	Control	May 23, 2023	Done	Mengjie Zhang
Beacon power control model	Urgent	SMPS	May 26, 2023	Done	Mengjie Zhang George Alexandros Melissii
PV panel characterisation	Urgent	SMPS	May 26, 2023	Done	Mengjie Zhang George Alexandros Melissii
Sensor interface			May 29, 2023	Done	
FPGA+ Camera Integration	Mid	Camera - FPGA	May 30, 2023	Done	Qidong Zhou Chenglin
Triangulation Deduction	Mid	Image Algorithm	May 25, 2023	Done	Chang
Triangulation Code Implementation (Python)	Mid	Image Algorithm	May 27, 2023	Done	Chang
Basic Algorithm	Mid	Pathfinding Algorithm	May 30, 2023	In progress	Chang
SQL Database	Chill	Database	June 1, 2023	In progress	Anastasis Varvarigos
Gyro scope control model	Mid	Control	May 23, 2023	In progress	Mengjie Zhang
Wi-Fi Comms	Mid	Comms	June 5, 2023	In progress	Anastasis Varvarigos
Front end UI	Chill	Frontend	June 10, 2023	In progress	Chenglin Qidong Zhou
Vector Simplified Mapping Algorithm	Mid	Image Algorithm	June 3, 2023	In progress	Chang Chenglin Qidong Zhou
Mechanical Design	Mid	Mechanical	June 3, 2023	In progress	Chang
Best Accurate Route Reinforcement Learning	Chill	Pathfinding Algorithm	June 7, 2023	Not started	Chang
Build/test motor control system	Chill	Control	June 7, 2023	Not started	Mengjie Zhang George Alexandros Melissii

Figure 1.3: Task allocation and division

A Gantt chart was used for team management which sets the deadlines for each people's work and speeds up different systems' integration. This ensures the most efficient design process.

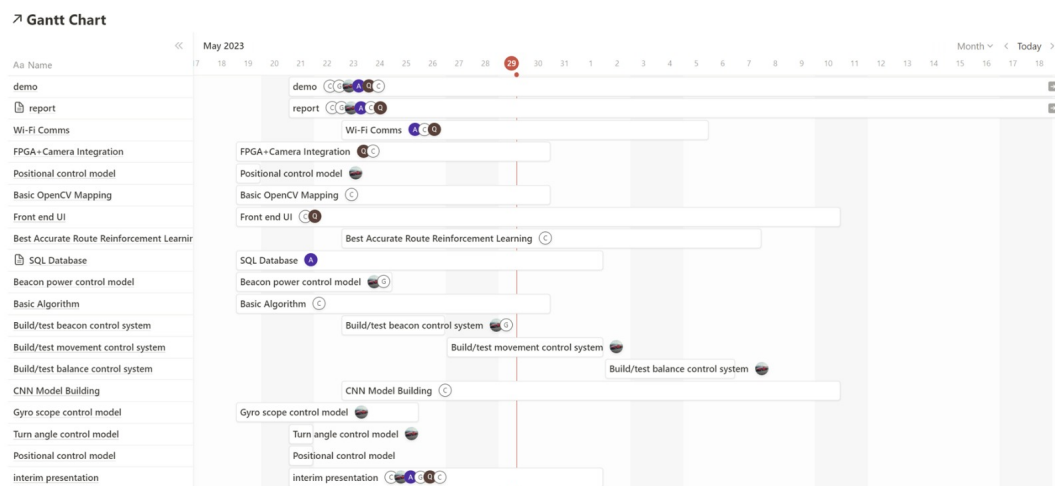


Figure 1.4: Gantt chart for project schedule

Chapter 2

Mechanical

2.1 Overview

The final mechanical design follows the demand given by the control team. It is compact, small, cheap in budget, and has a decent amount of redundancy. The robot is designed with 2 layers and 6 surfaces for mounting. The dimension is 130mm x 90mm x 200mm. The cost of manufacturing of the chassis is £0.00 with all the given parts, laboratory scraps, and screws and nuts.

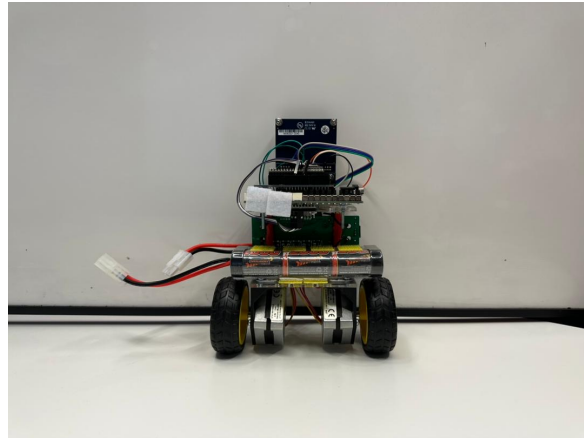


Figure 2.1: Final Robot Assembly

The structure of the robot consists of two major layers and one minor mounting sides. The bottom layer is used as the mounting plate for the two motors with original mounting plate and wheels, the battery, and the structural support for the upper layer. The upper layer is used as the mounting plate for the FPGA, esp32, camera, and the accelerometer/gyroscope. There are two pillars connecting the upper and lower plate. This pillar is also acted as the mounting space for the motor control board using zip ties.

2.2 Manufacturing Constraint

The robot is manufactured using laser cutter, using scrapped 3mm clear acrylic. There are several considerations for this decision apart from the basic requirements of having all components mounted on one robot.

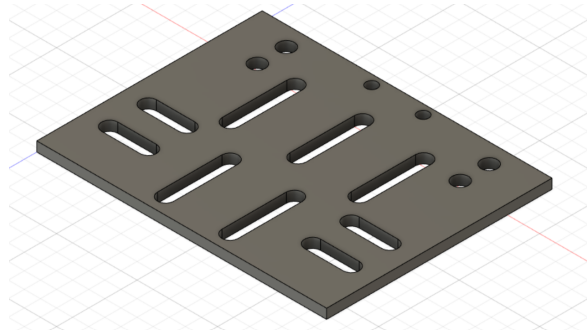


Figure 2.2: Final Plate Design

Budget

The preliminary consideration pointed out that the cost for extra external sensors, motors, etc. will be high. Thus, the cost of chassis manufacturing needed to be reduced to a minimum. The lab provided several means to cut cost. 1) The fixing pieces are free. 2) The additive manufacturing (3D printers) and materials are free. 3) The acrylic scraps can be free if it was bought before. Due to the second consideration, it was decided to use laser-cut acrylic as the major structure. One challenge was the size of the scraps. The pieces were thoroughly cut and there was barely any space on it. Design choices were made that the plates would be small enough to be cut from the scrap acrylics. Thus, the 70mmx90mm size was determined.

Fast Iteration Time

Due to an increased number of usages of the 3D printers in the lab, it was impossible to have the base chassis iterated in a daily basis. Moreover, due to the nature of additive manufacturing, it was impossible to get pieces printed quickly as well. However, the laser cutter was rarely used and available almost all the time and it would only take it minutes to produce a piece. Ideally, the iteration speed can be increased to a ridiculous pace where pieces can be produced every 5 minutes. This provided a huge advantage to try the new designs with control team and finalize the mechanical design.

Dimension

The robot is required to be able to fit in and manoeuvre through the maze, which limited the projection area of the robot to 300mm x 300mm. Another limitation is provided by the control team that the rover had to be small to 150mm maximum in length. Moreover, in order to utilize the scrap, the plate length was further set to 90mm maximum which resulted in a length of 130mm maximum. The height limitation

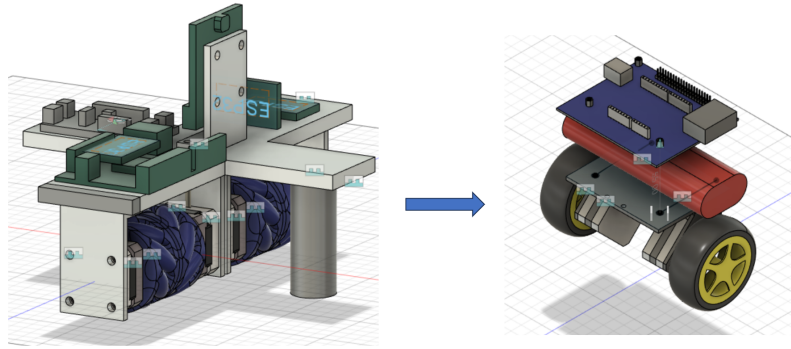


Figure 2.3: Design Iterations

was determined by the camera viewability. A series of experiments were conducted to compare the camera height to the viewability of the camera as in figure 2.4.

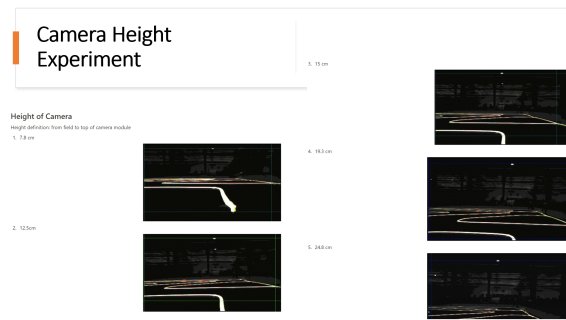


Figure 2.4: Camera Height Experiment

Redundancy

Though the chassis was iterated many times, reusing the parts was still desirable due to budget concerns (limited scraps). Every iteration provided more than enough mounting surfaces for future use and expansion. The upper board from the second iteration was designed to be interchangeable with the lower board in case there were any necessity to mount more components on the lower board.

Stability

A stable testing environment is required by the control team. The components need to be fixed to one place and have minimum wobbling. Thus, the robust fastening measures are considered. The plates were designed to maintain multiple fixturing positions to hold stability.

2.3 Iteration History

The mechanical design has two major iterations with minor iterations throughout. The first iteration involves taking account for a leaping goal with reusability, redundancy, and extendibility. The second iteration is an overhaul based on the first iteration and follows the additional constraint and the demand from control team.

The first major iteration was a long chassis board design. The ultimate goal for this chassis was to have two parallel mecanum wheels sets. Mecanum wheels are able to provide a diagonal direction vector. With four of the mecanum wheels, it is possible to have an omni-directional movement. The idea was to put two of mecanum wheels parallel to each other as one wheel. One wheel will be driven by two motors to control the different speed for each mecanum components of the wheel. It would provide a lot of control challenges. However, it reduced the complexity for the future algorithm.

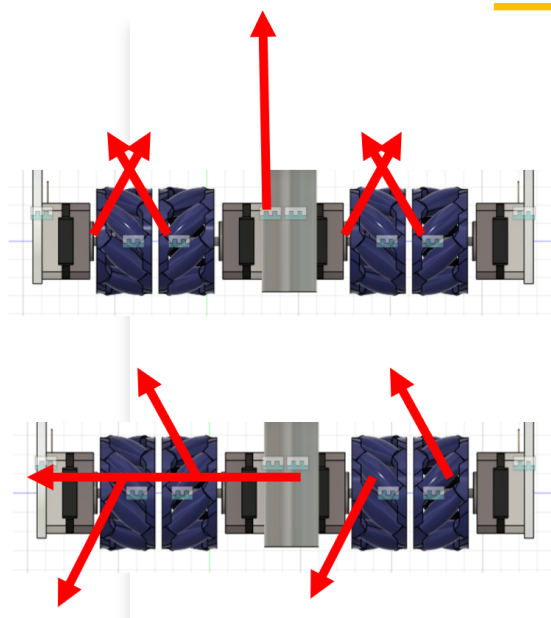


Figure 2.5: Mecanum Wheels

The second major iteration was a short chassis board design which was the final design. After the control team was experiencing difficulties of the basic balancing. The mecanum wheel plan was abandoned. It was required by the control team that the chassis as to be small as well. Thus, this version of the chassis design was introduced and finalized after several minor iterations.

Chapter 3

Control System

3.1 Sensing

3.1.1 Acceleration, velocity and distance

The MPU6050 sensor contains a significant stationary error which need to be removed for a reliable function. The most influential variable on distance is the x-direction acceleration. It is important that the acceleration is calibrated as close to perfection as possible as the error produced by the acceleration measurement builds up overtime and will get amplified at the distance measurement end.

There are two method to calibrate acceleration:

Upper-lower threshold method

This method involves sampling a large set of the stationary error of the acceleration during a given time while the sensor is kept motionless, and then identifies the largest and smallest error occurred: ub and lb . Afterwards, if acceleration lies within the threshold boundary, then the reading is purely due to stationary noise and deem the robot as stationary.

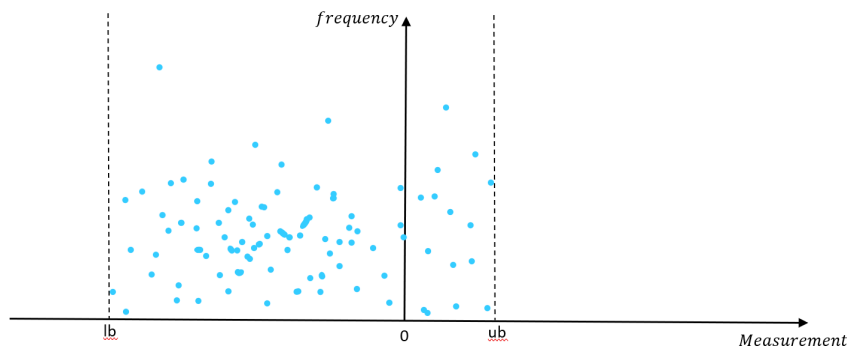


Figure 3.1: The error frequency diagram

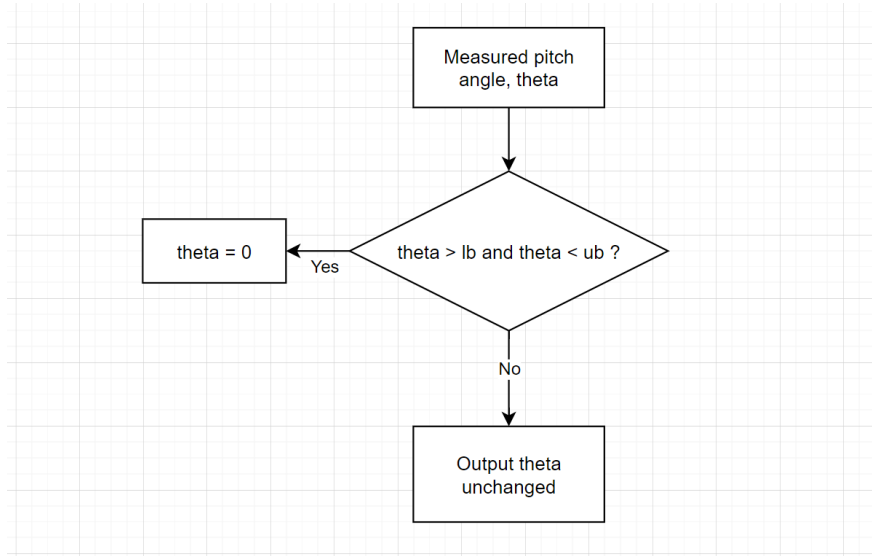


Figure 3.2: Threshold method implementation

Average subtraction method

This method involves sampling a large set of the stationary error of the acceleration during a given time while the sensor is kept motionless and calculate the average error, $\bar{e}$. Then subtracts the average error from each acceleration measurement: $\bar{\alpha} = \alpha - \bar{e}$. Afterwards, a large set of the acceleration was sampled with their average offset being removed and the largest absolute deviation of the acceleration from the 0 point was identified, $|\bar{\alpha}_{max}|$. Hence for $\bar{\alpha} < |\bar{\alpha}_{max}|$, the acceleration was set to 0 and for $\bar{\alpha} > |\bar{\alpha}_{max}|$, acceleration = $\bar{\alpha}$.

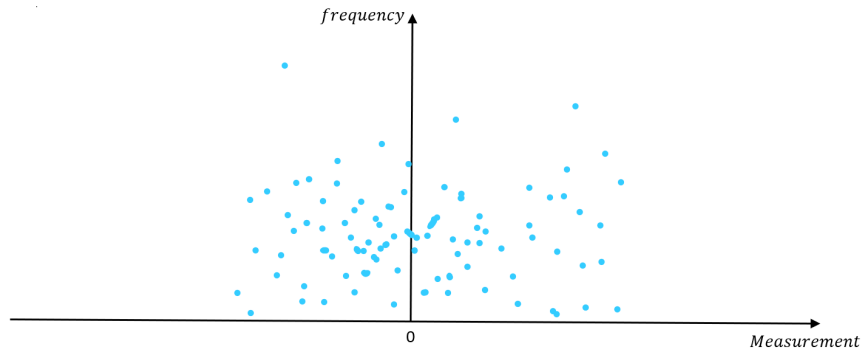


Figure 3.3: Bringing the error cluster to about its average position

Discrete integration and integration errors

From the previous equations, all that's needed to find distance is to integrate twice the acceleration. The general approach is to calculate the time elapsed between each measurements and then multiple the newly measured value with the time and add it

to the previously measured value.

$$v[n] = \alpha[n - 1] + \alpha[n]dt$$

$$x[n] = v[n - 1] + v[n]dt$$

However, careful measurements need to be taken when dealing with errors arise from discrete integration. And the 2 methods previously described has different levels of error for each.

For the first method, it is effective in removing the acceleration noise but it also leads to a big problem.

Since the error cluster of the acceleration lies only on one side of the graph(1), the acceleration required to activate a non-zero detection is different for forward and backward acceleration. This means that the accumulation of acceleration for positive velocity and negative velocity are different, hence there's a significant offset error on top of the other errors. For this reason, it will not be used.

The second method eliminates this offset error by bring the error cluster back to the 0 point using average subtraction, and instead of having two thresholds with different magnitudes, only one threshold is used for both positive and negative magnitudes. This ensures that the same amount of acceleration is added or subtracted from the velocity.

Unfortunately, due to a finite sampling time, there's still a small integration error after each velocity transition; moreover, due to the limitation of the ESP, it is not possible to increase the sampling frequency beyond 16MHz, in fact, due to the execution delay of the code, the highest sampling frequency of the sensor is around 338 Hz. Also, because the acceleration is non-linear, the area omitted in the cutoff region is different for positive and negative accelerations, resulting a tiny offset error. Even though the error is negligible on its own, it will build up overtime and eventually became so large as to make the velocity and distance measurements invalid.

In conclusion, methods involving calibrate the sensor at the very start of the operation once and for all is not reliable in the long run and further calibration techniques need to be developed. One technique is the in-operation calibration where the measurements are adjusted at any time as we see fit so that the error is not carried forward into the next round of integration process.

Conditional calibration

One of the in-operation calibration method is the conditional calibration. In this method, the sensor is calibrated once there's a measurable error being detected. For instance, when the robot first stops after moving for a while, its velocity may not return to 0 exactly but being very close to it. In this case, it's certain that there's an integration error during the process and can therefore force the velocity to be 0. To do

this, extra caution is needed since there could be a situation where the velocity is small but kept varying really fast, therefore, an additional constraint of needing $\bar{\alpha} < |\bar{\alpha}_{max}|$ is applied so that it's certain that the robot has stopped.

The above methods still subject to that the rate of change of the velocity is not too large so that small values can be effectively captured inside the cutoff region. Fortunately, the robot was not purposed to run at high speed thereby justifies the usage of the above method for the final approach of distance measurement.

3.1.2 Pitch angle

Similar to acceleration, angular velocity measurements need to be calibrated. The method of calibration will be the *average subtraction method* which was previously proven to be more reliable.

Discrete integration and integration errors

The integration process for the pitch angle follows the same procedure as before:

$$\theta_y[n] = \omega[n-1] + \omega[n]dt$$

Similarly, the discrete integration for angle also give rise to accumulative errors. Although for relatively small angular velocities, the angle can be calibrated back to the equilibrium point using *conditional calibration*, the bandwidth requirement of the controller would likely mean that oftentimes the conditional calibration will not respond and still leads to an ever-increasing/decreasing angle measurement, which in the light of designing a self-balancing rover, is devastating.

This is due to the method for measuring distance has a fatal flaw, it is only capable of measuring values relative to a predefined reference - a.k.a. *relatively* accurate. Once the reference point is lost, the sensor is unable to produce any valid measurement. Moreover, calibration at the starts takes a long time and is easily affected by disturbance, making the rover non-practical.

Trigonometric method

To solve the above problem, a method which can achieve *absolute* accuracy is needed and one of the existing way of doing so is the trigonometric method. This method calculates the current angular position of the sensor based on the gravitational acceleration at x, y, and z axis.

$$\theta_y = \arctan\left(\frac{-a_x}{\sqrt{a_y^2 + a_z^2}}\right)$$

Calibration is needed to eliminate the stationary error of the accelerations, this can be easily achieved by calculating the standard deviation of the accelerations w.r.t $g(9.81)$.

Since this method is absolutely accurate, there's no need for calibration at the start of the operation, but nonetheless, it has its drawbacks. As the MPU6050 sensor has high frequency static noise, the accelerations will be oscillating rapidly around an average value and so does the calculated angle. Because of this, a way to eliminate this oscillation is required.

Kalman filter

The above two methods each has its advantages and disadvantages. The integration method is smooth but inaccurate while the trigonometric method is accurate but noisy. However, the accuracy of the latter method can be used to compensate for the inaccuracy of the former method. Hence the use of complementary filters are the most suitable choice here and a widely known one is called the "*Kalman filter*".

For every iteration, first carry out discrete integration as usual. Then, calculate the

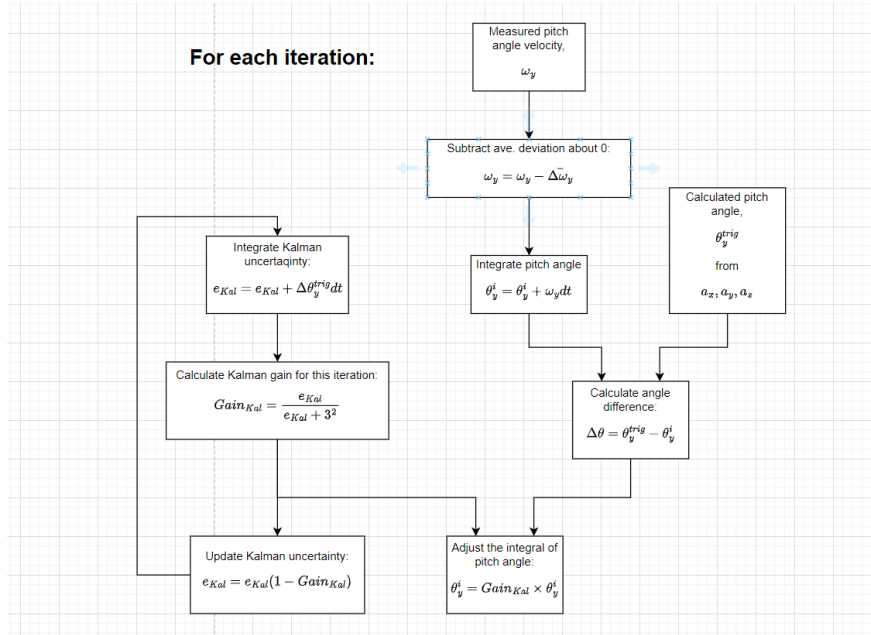


Figure 3.4: Kalman filter implementation

accumulation of the Kalman uncertainty over time. Afterwards, calculate the Kalman gain and multiply the gain with the difference between the trigonometric measurement and the integration result. Finally, update the uncertainty using the complement of the gain.

The Kalman uncertainty measures how noisy the trigonometric measurements while

the Kalman gain measures how much the program should alter the integration results w.r.t. the trigonometric measurements. As can be seen from the diagram above, the larger the uncertainty or the larger the standard deviation, the smaller the gain, meaning the trigonometric measurements are not as "trust-able", so the program should stick more to the integration results, the opposite is true if the uncertainty or the standard deviation is small, it would be more inclined to change the integration results basing on the trigonometric measurements.

The following figures shows a comparison of the angle measured using integration, trigonometric and Kalman filter method respectively.

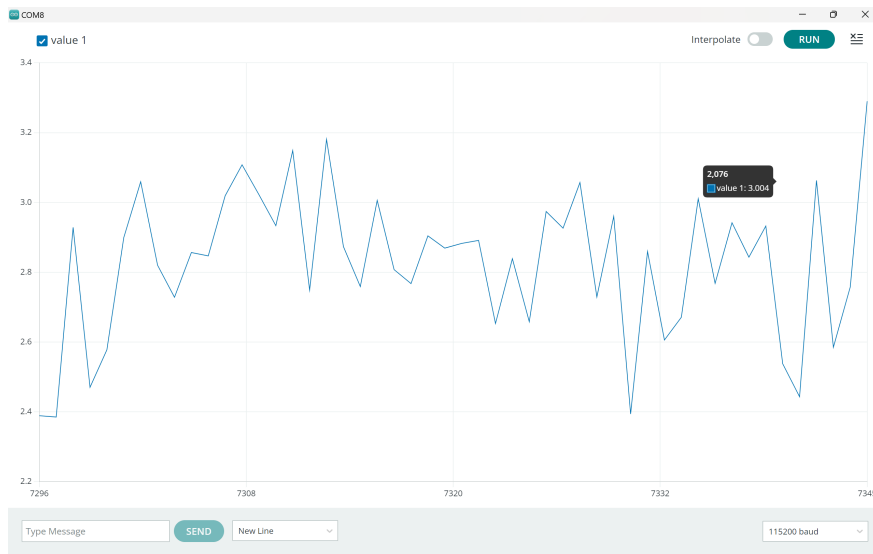


Figure 3.5: *Pitch angle stationary measurement using trigonometric method only*

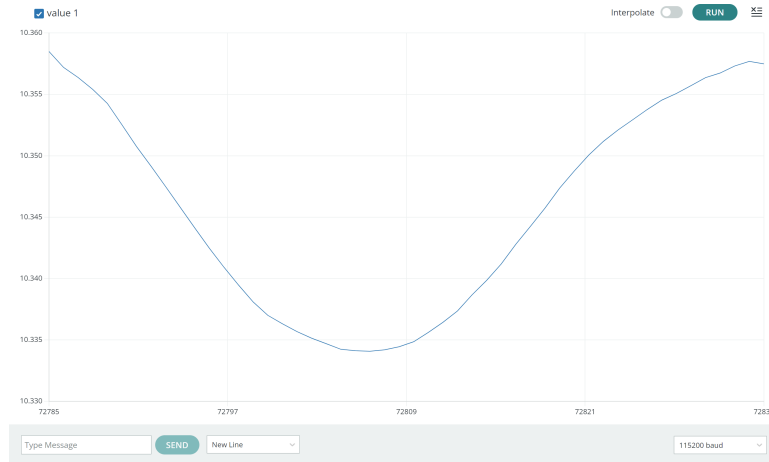


Figure 3.6: *Pitch angle stationary measurement using Kalman filter*

3.2 Balancing

3.2.1 Design concept

Design choice 1

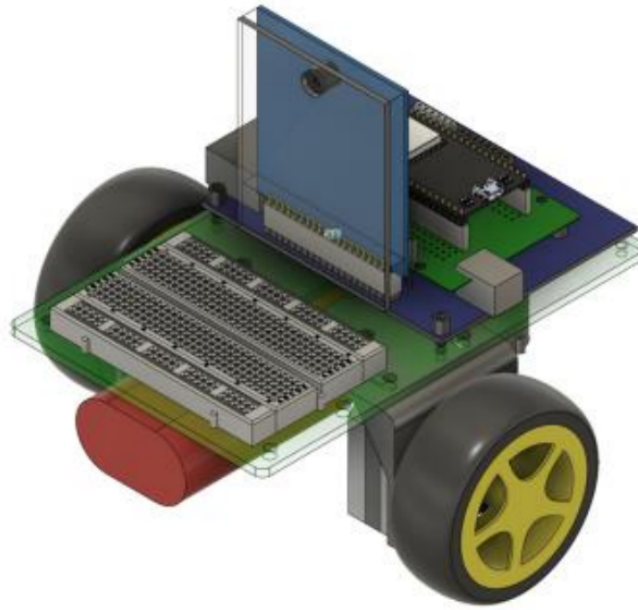


Figure 3.7: "*Horizontal lying design*"

The first model used for design and testing purposes is the *horizontal lying design*.

The horizontal lying rover has the advantage of being more stable and does not require much torque to keep it balanced while the disadvantages being that the pitch angle changes slowly therefore the controller will also react slower to disturbance. Moreover, its occupied area is much larger and it's less mobile, meaning it is not practical for exploration of complex passages.

For those reasons, design choice 1 was used mainly for debugging and early testing.

Design choice 2

The second choice is the *partial vertical standing design* and is developed under the modification of the first choice where the motor is fixed at the bottom of the chassis.

The second choice offers a more compact and mobile solution to the rover design but due to the limitations of the mechanical components, it can only balance at an angle approximately 76 degrees. This makes the robot much less stable than it could

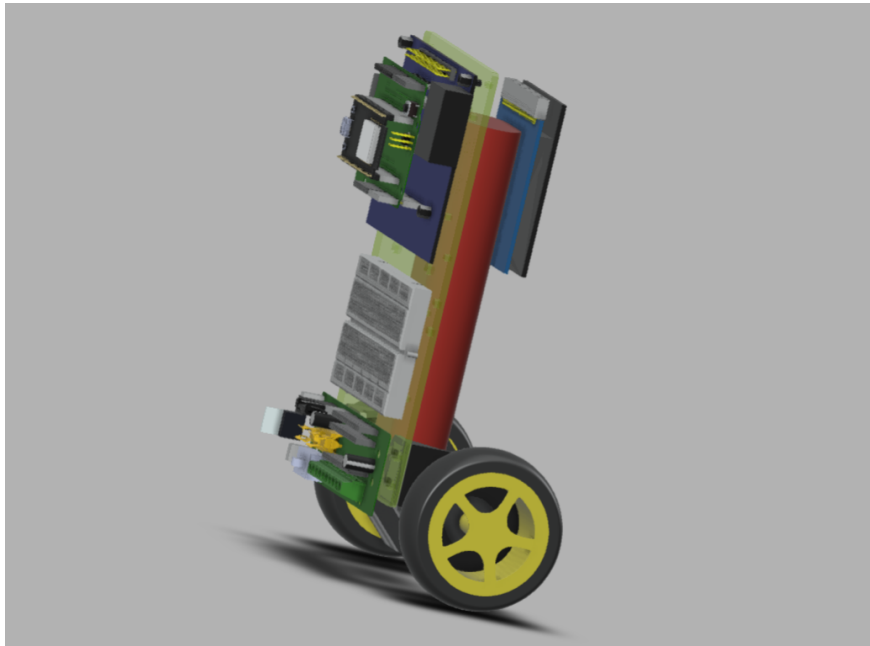


Figure 3.8: *"Semi-vertical standing design"*

be. This also means that it would limit the camera angle and its field-of-view, making navigation harder than it should be.

Hence the second design choice is used for testing the balancing ability of the designed controllers.

Design choice 3



Figure 3.9: Inspiration for design choice 3

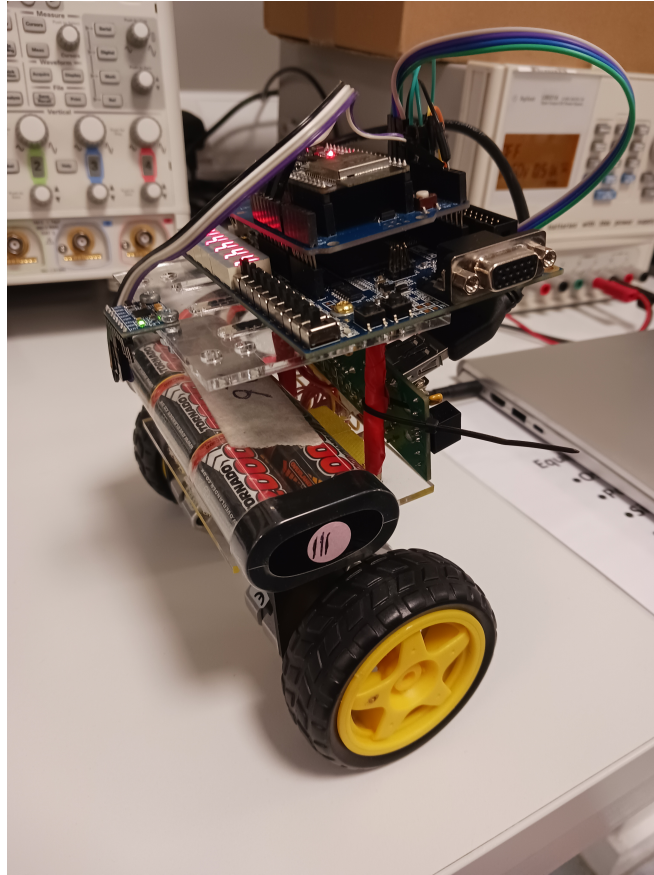


Figure 3.10: *"Vertical standing design"*

The final choice offers more stability, compactness, mobility and usability than the previous two, making it perfectly suitable for explorations and navigation. By stacking core components together between acrylic boards, the robot's spatial efficiency and modularity can be increased. It also greatly reduced its surface area as seen from a three-view drawing, making it able to pass through smaller spaces. Moreover, it has good packaging efficiency when it comes to mass producing the robot, making it more commercially viable.

3.2.2 Modeling and simulation

Simple MATLAB model

The first model was made before the full dynamic analysis of the rover was released. It attempts to use basic physics to describe the pitch angle dynamics of the model.

$$T = J\alpha$$

$$\dot{\omega} = \alpha$$

$$\dot{\theta} = \omega$$

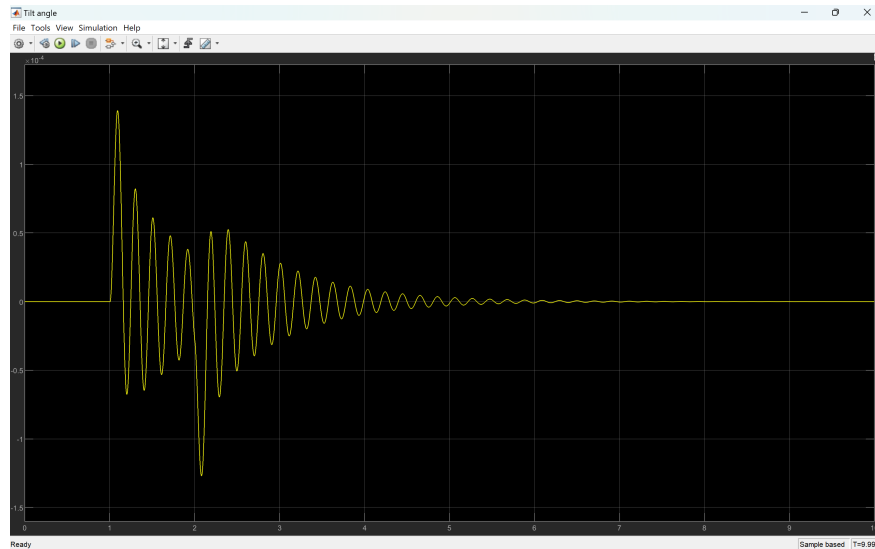


Figure 3.11: Simulated result for pitch angle control

As can be seen from the simulation, the change of the robot's pitch angle is effectively suppressed by the counter torque produced by the controller, meaning that theoretically the controller would balance the rover well.

An important fact is that to produce a counter torque on the rover, the wheels must rotate at the same direction as the tilting of the robot.

3.2.3 Real time sensor measurement

The key to achieve robot balancing is to ensure a stable and accurate measurement of the pitch angle at all times during operation so as to maximise the effectiveness of the controllers.

3.2.4 Controller design

The key of balancing the robot is to design a suitable controller, and, since the robot's dynamics changes between different designs, the controller design will be depending on those design choices.

Feed-forward controller

One of the feed-forward control strategy considered was the *fuzzy logic control* and was tested on the *horizontal lying design*.

The idea of fuzzy logic control is to simplify the control as much as possible since it uses only "common sense" to balance the rover. First, categorise the states of the

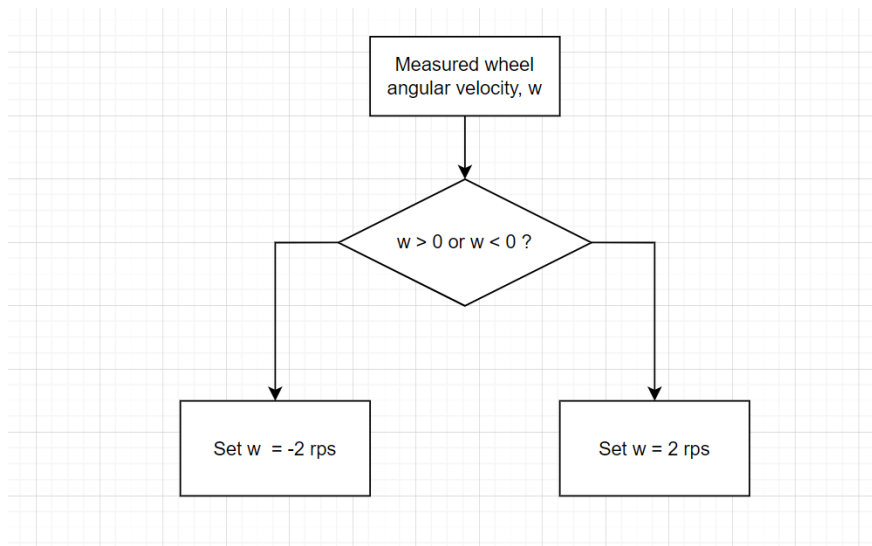


Figure 3.12: Fuzzy logic control implementation

rover into different levels, e.g. slightly imbalanced, moderately imbalanced and very imbalanced. If the angle is too steep, rotate the wheels faster and if the angle is not too steep, rotate them slower.

Unfortunately, under extensive testing and adjustment, this control method is incapable of achieving balancing and was therefore scrapped.

Single loop feedback controller

Among all the closed loop controllers, the single loop controller is the simplest and easiest to implement. In our design, the *proportional feedback control* method was tested on the *horizontal lying design*.

2 possible control loops can be applied, the pitch angle control loop and the angular velocity control loop. The gain can be adjusted to make the robot stable.

However, after testing and tuning, the controller failed to balance the robot.

Single loop feedback controller with gain scheduling

Although the previous controller didn't function as expected, it was seen to have the potential of reversing the tilting effect, therefore some extra adjustment can be implemented in an attempt to stop it from over reacting since it would be ideal to have a simpler controller.

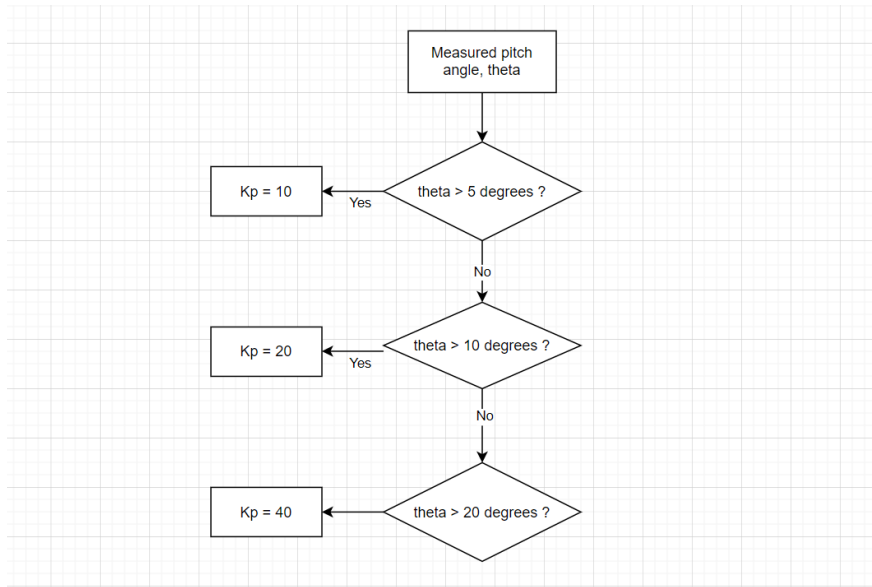


Figure 3.13: Gain scheduling implementation

Gain scheduling is a method in which, instead of having a constant K_p for the entire operation, the program adjusts its value at different operating points. In this case, when the error is small, the gain is small so that the controller doesn't overdo; whereas if the error is large, the gain is made larger to restore the balance.

However, after testing and tuning on the *horizontal lying design*, the controller still failed to achieve the desirable performance, therefore the single loop control strategy was proved to be impractical.

Duo loop feedback controller

The previous failures call for a much more robust controller design, and the first one considered was the *duo loop feedback controller*.

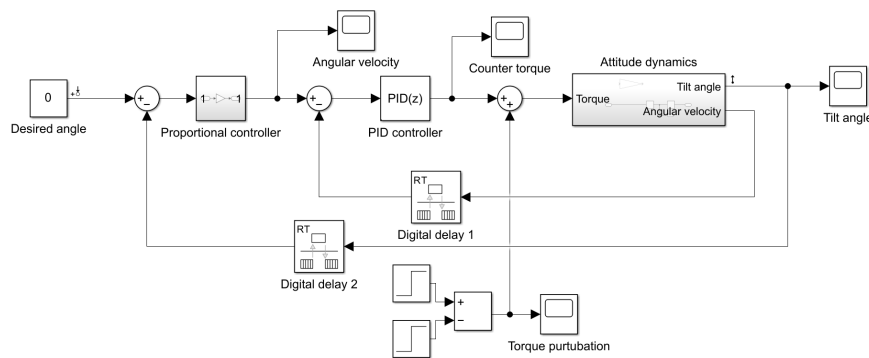


Figure 3.14: Pitch angle controller using a proportional gain and a PID controller

The first duo loop controller implemented uses 2 proportional feedback controller and after adjusting the gain, it's able to balance the *horizontal lying design* with small marginal stability. However, it wasn't able to achieve long term balancing on the *semi-vertical standing design*.

In order to deliver better balancing performance, a PID controller is used in place of the proportional controller pf the pitch angular velocity. Initially, set K_d to 0 so the controller is only a PI controller.

Despite this, the controller is able to balance the *partial vertical standing design* well with near asymptotic stability.

Motor driver

One problem still remains even after the most sophisticated controller has been used, is that at the equilibrium position, the robot will oscillate back-and-forth as well as producing loud noise, making it less stable and more prone to tilting while moving and turning. This is due to the motor's step resolution is small and the driver is only able to produce one step at a time using the basic settings.

To solve this issue, it's crucial to perform *microstepping* so that the wheels do not over react and the precision of the distance travelled is higher.



MS1	MS2	MS3	Resolution
LOW	LOW	LOW	Full Step
HIGH	LOW	LOW	Half Step
LOW	HIGH	LOW	Quarter Step
HIGH	HIGH	LOW	Eighth step
HIGH	HIGH	HIGH	Sixteenth Step

Figure 3.15: Pin configurations for microstepping on A4988

(Figure 3.16) shows the motor driver configuration needed for microstepping, noting that additional soldering needed to be done with extra pin arrays.

However, using microstepping means that a greater number of iteration loop is required to achieve the same angular speed as the full step version and since the program

contains sensor measurement and PID controller execution for each loop, the speed of the wheels is significantly slowed down, leading to reduced bandwidth and the inability to stabilise the robot.

One possible method of resolving this problem is to use multi-threading, but since the ESP32 DEV module already contains 2 cores, the sensor measurement task can be assigned to one core while the motor control task assigned to the other.

After implementing the duo-core configuration, the wheel speed has significantly improved; although it can only rotate at half the speed as before, it's fast enough to achieve a close loop stable bandwidth.

3.3 Linear motion

3.3.1 Design concept

Duo wheel system

A Duo wheel system allows only forward and backward linear motion, therefore additional steering functions need to be added to achieve flexible movement.

The advantage of using this system is that it's simple for the design process and allows to wheels to rotate at full speed thereby offers a good power efficiency while the down side is that it offers only 1 degree of freedom when moving in a straight line.

3.3.2 Modeling and simulation

Physics

The linear motion of the robot is dictated by simple kinematics:

$$\dot{x} = v$$

$$\dot{v} = a$$

$$v = \omega R$$

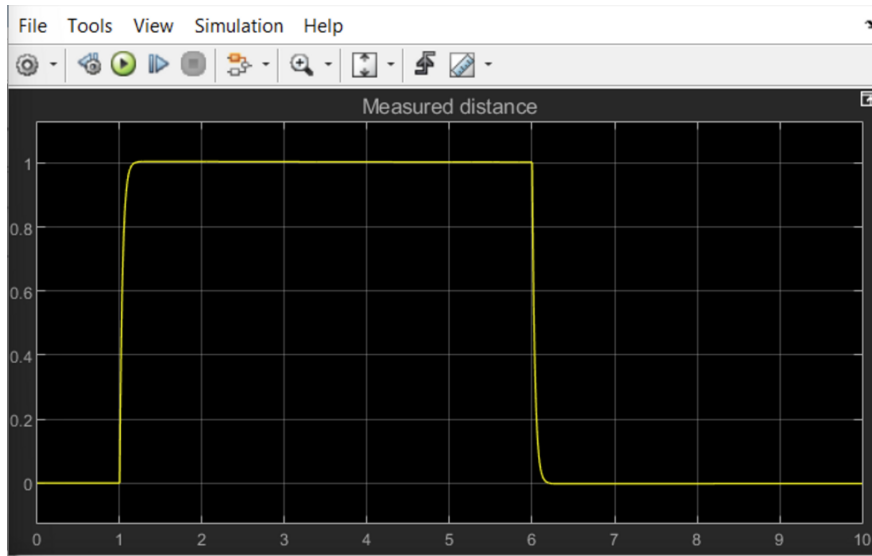


Figure 3.16: Simulated result for distance control

3.3.3 Dead reckoning

Wheel revolution counting

From simple mathematics:

$$distance_{travelled} = \frac{numberofsteps}{stepsperrevolution} \times wheelcircumference$$

The Accelstepper library function **currentPosition()** returns the total number of steps executed by a motor. The above formula can be used to calculate the total distance covered and since the wheel is non-slippery, this method of dead reckoning gives a high accuracy.

Cascade controller

Several attempts had been made to make the robot move including stepping the motor after balancing has been executed but since the pitch angle controller is robust enough to always cancel any movement made by the motors, a more robust distance controller need to be implemented.

The first attempt was to use a single loop proportional controller which measures the distance error and apply the corresponding tilt. But this method is unreliable as distance increases, the tilt has to increase, which leads to the robot tipping over. Therefore a duo loop distance controller is used.

The robot is able to achieve precise movement with stable velocity.

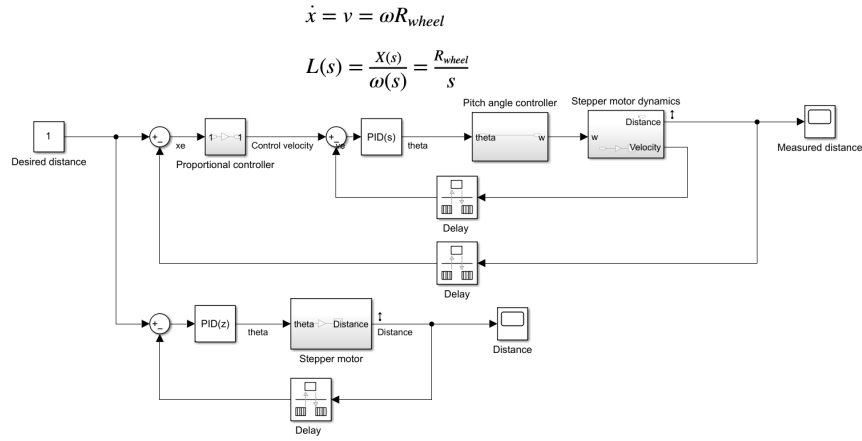


Figure 3.17: Duo loop controller for distance and velocity

3.4 z-x plane turning

3.4.1 Design concept

There are 3 possible ways of turning for a duo wheel system:

Centred rotation

Centred rotation results in 0 displacement.

Pivot rotation

Pivot rotation results in a non-zero displacement.

Differential rotation

Differential rotation results in a non-zero displacement.

3.4.2 Modeling and simulation

The kinematics of the robot's rotation for the 3 different scenarios above:

scenario 1:

$$\dot{\theta} = \frac{360\omega R}{\pi l}, \text{ Power} = T\omega, \dot{x} = 0$$

scenario 2:

$$\dot{\theta} = \frac{180\omega R}{\pi l}, \text{ Power} = T\omega, \dot{x} = \frac{\omega R}{2}$$

scenario 3:

$$\dot{\theta} = \frac{180(\omega_2 - \omega_1)R}{\pi l}, \text{ Power} = T(\omega_1 + \omega_2), \dot{x} = \frac{(\omega_1 + \omega_2)R}{2}$$

3.4.3 Dead reckoning

It's required to discretize the kinematics equations so that it's suitable to be used in the Arduino Accelstepper library:

scenario 1:

$$\theta = \frac{360 \times \text{number of steps}}{\text{steps per revolution} \times \pi \times \text{axial length}} \times \text{wheel circumference}$$

scenario 2:

$$\theta = \frac{180 \times \text{number of steps}}{\text{steps per revolution} \times \pi \times \text{axial length}} \times \text{wheel circumference}$$

$$x = \frac{\text{number of steps}}{2 \times \text{steps per revolution}} \times \text{wheel circumference}$$

scenario 3:

$$\theta = \frac{180 \times (\text{step ratio}_2 - \text{step ratio}_1)}{\pi \times \text{axial length}} \times \text{wheel circumference}$$

$$x = \frac{\text{step ratio}_1 + \text{step ratio}_2}{2} \times \text{wheel circumference}$$

3.4.4 Turn angle controller

For better turning performance, we will choose pivot turning method as it can make the robot aligned with the entrance. For the turning mechanism, a speed offset is added to both motors, the motor's balancing algorithm will try to achieve the offsets; obviously since both motors are rotating in the opposite directions, it will never achieve an equilibrium point, the motor ends up turning constantly.

Since the turning process of the robot is much more stable than moving, we only need to use a single loop to achieve good accuracy and stability. The dead-reckoning method was proved to be accurate as well after field testing.

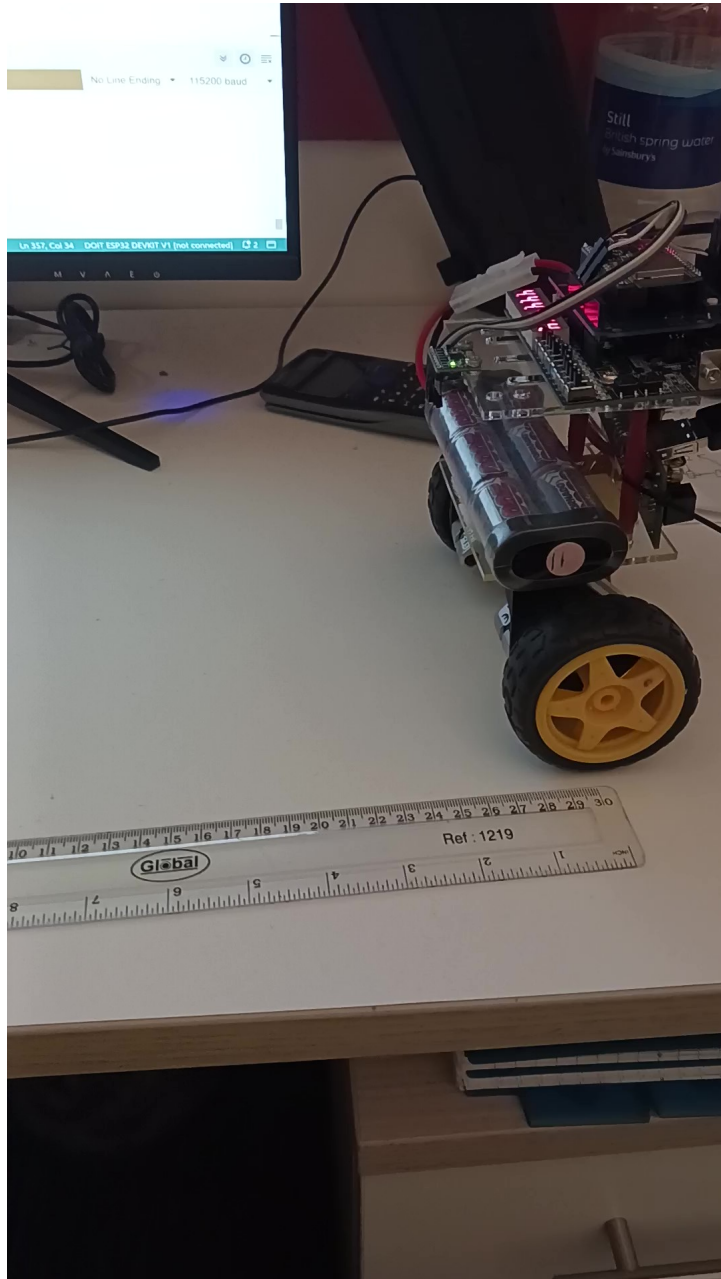


Figure 3.18: Movement test 1

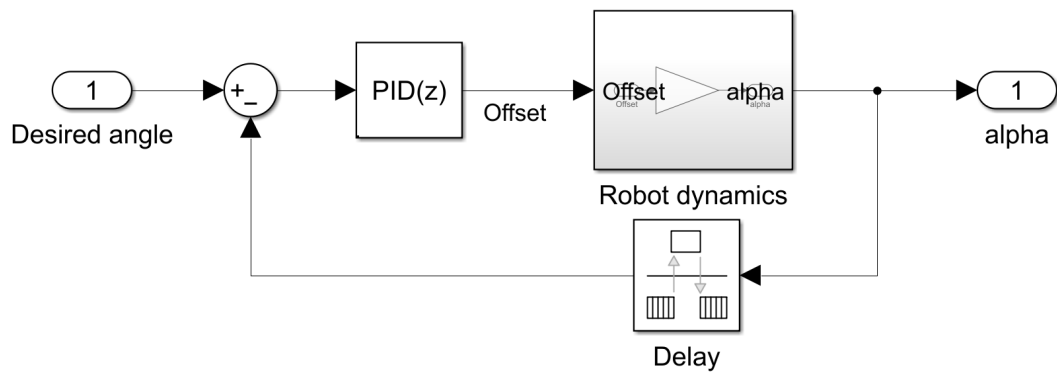


Figure 3.20: Single loop feedback controller for turning angle

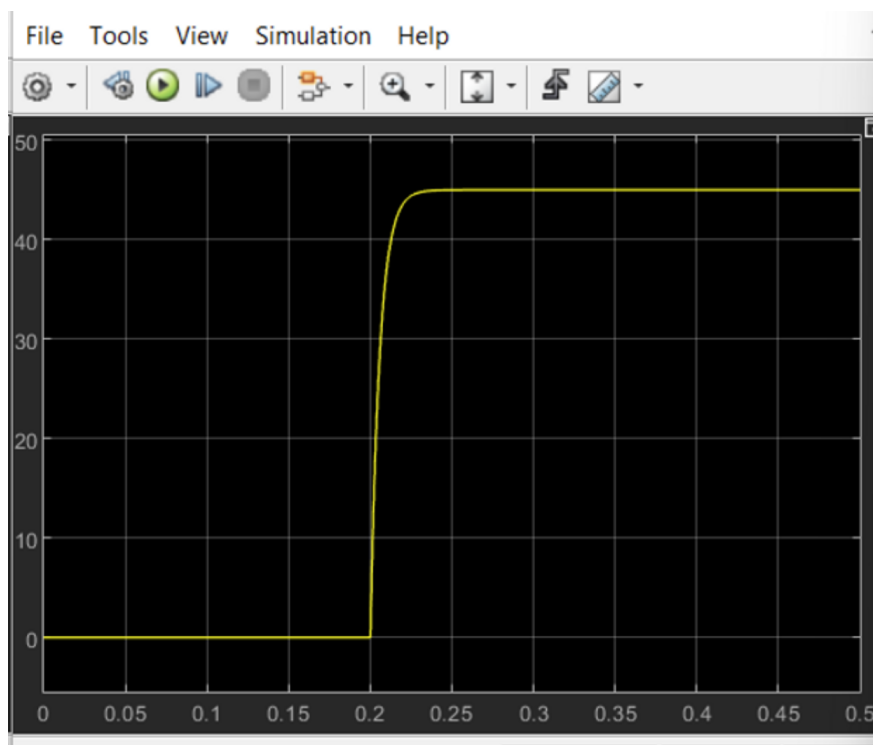


Figure 3.21: Simulated result for turning angle control

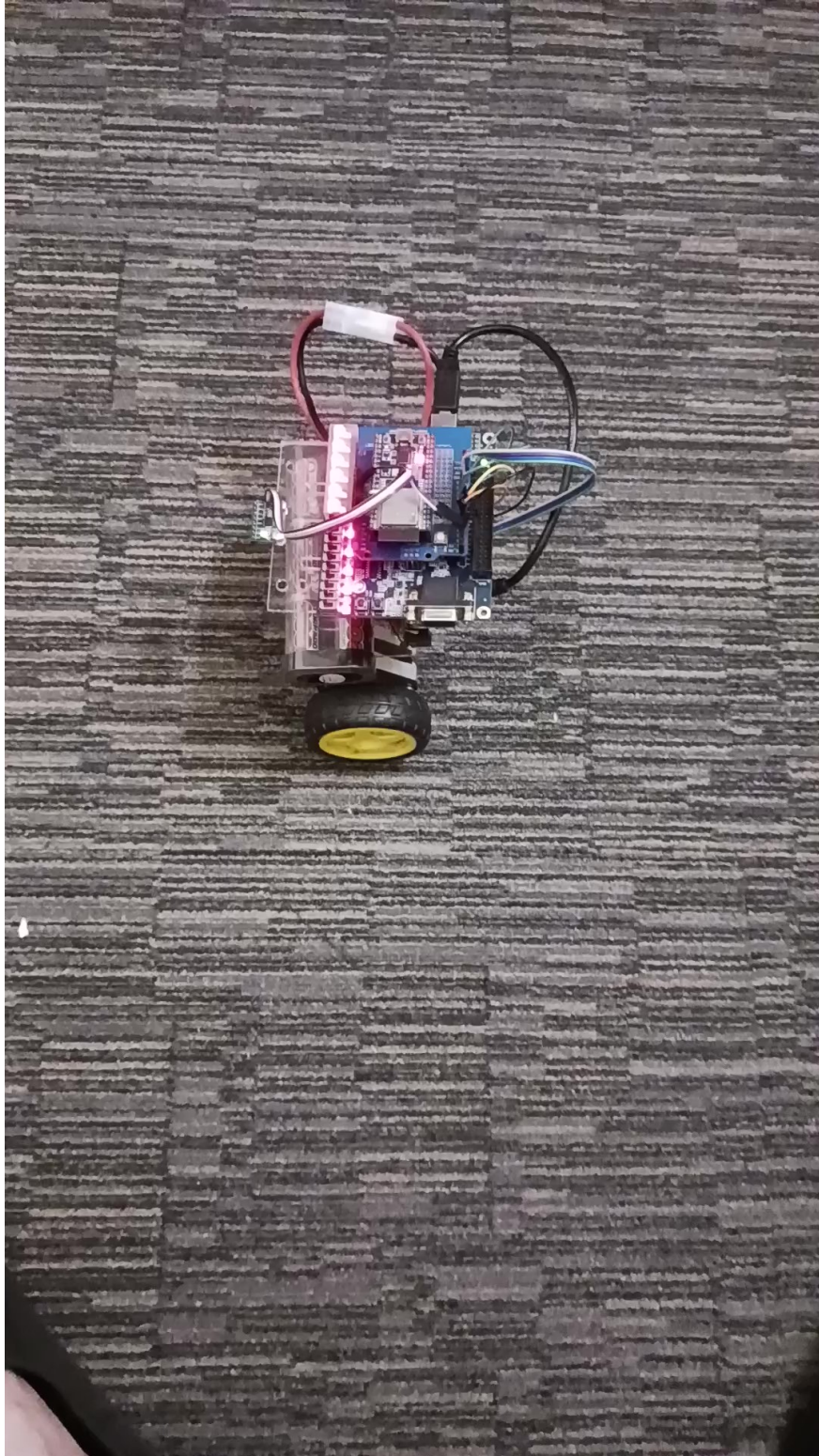


Figure 3.22: 90 degrees turning test phase 1

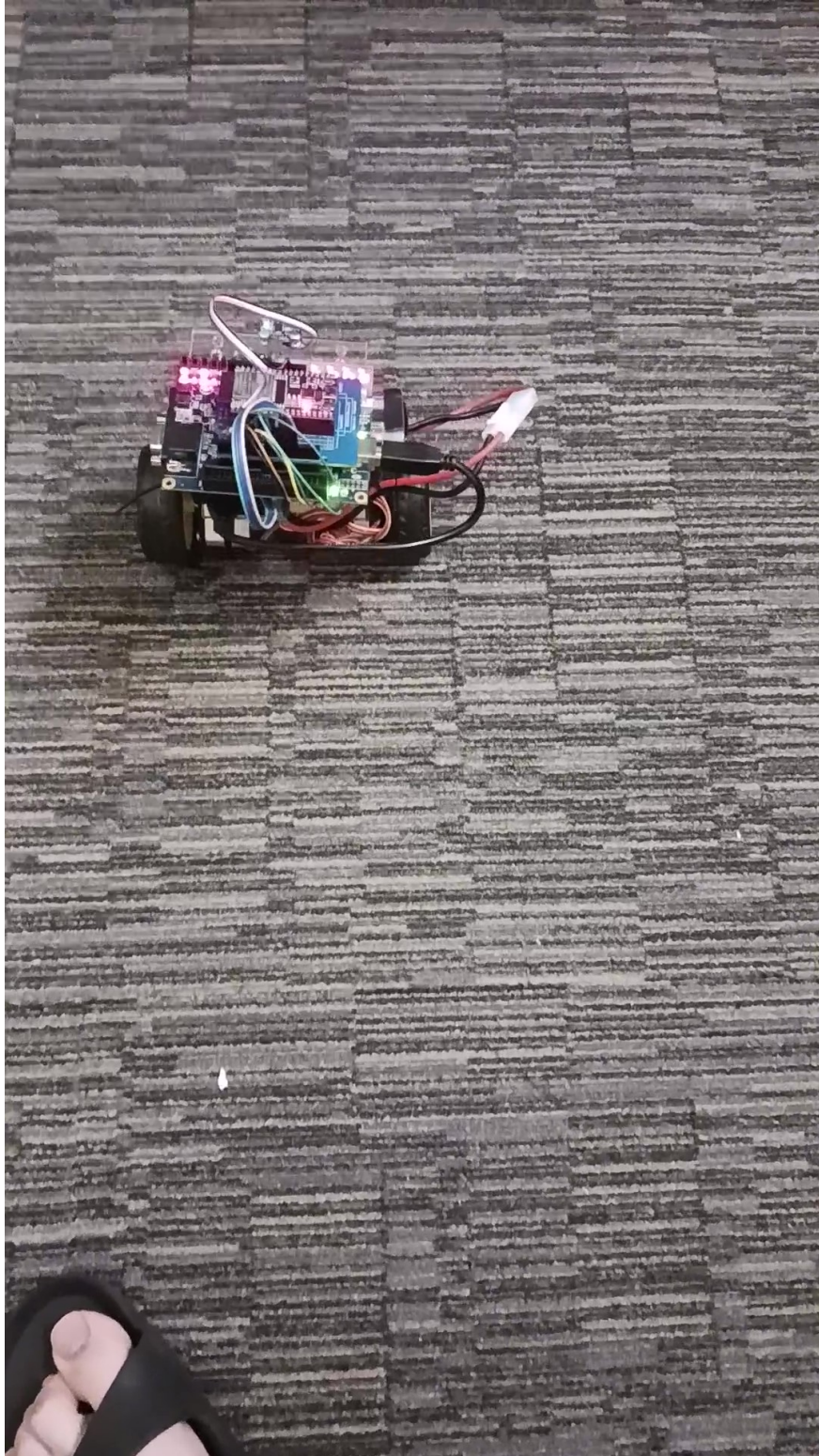


Figure 3.23: 90 degree turning test phase 2

Chapter 4

Power System

4.1 Problem Definition

For the rover to map and navigate the maze it is going to need three reference points, which are represented by the different colored LED beacons. The assigned task is to supply these LEDs with enough power from a solar or emulated configuration by balancing the supply and demand of power even at times of limited supply. The energy subsystem consists of three PV panels, three LED beacons, 4 energy conversion modules and a supercapacitor for energy storage.

4.2 Top-level functional design

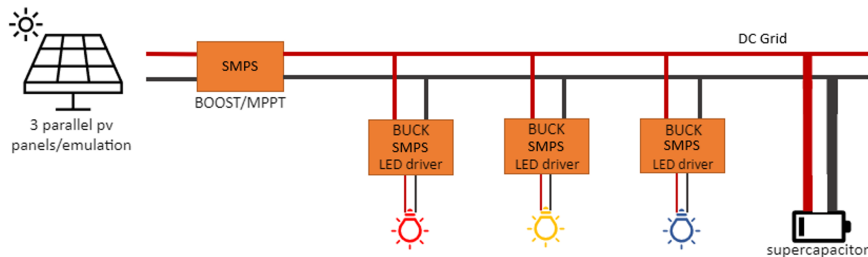


Figure 4.1: Optimum design for power supply system

Energy from the sun is received by the solar panels and is transformed into electrical one to further supply the Boost SMPS, which is programmed with a MPPT algorithm to draw maximum power from the panels and with a safety measure of turning the SMPS off for more than 17V to ensure safe operation and prevent overvoltage of the Boost SMPS output and LED drivers input. The output voltage of the Boost supplying the DC Grid is stepped down by the Buck LED drivers to the operation voltage of the LEDs which is about 3V. The Buck LED drivers are programmed to be turned on only when the minimum input voltage across them is at least 7V which is the minimum voltage limit for safe operation as the Raspberry Pi Pico W soldered to them needs 3.3V to work properly. In the case of limited or insufficient energy supply from the PV

panels the supercapacitor comes into play to hold the beacons on for a couple more seconds up to the point that the voltage drops to 7V or the energy is sufficient again for operation without it.

4.3 Implementation and Design choices

The configuration of the solar energy system was chosen to be three PV panels in parallel supplying in total about $V_{OC} = 5V$ and $I_{SC} = 0.69A$. The reason for this choice was that the SMPS used would be Boost OL non-synchronous so the voltage wouldn't be too high but enough for the Boost to step it up to the required minimum LED driver input and because the LEDs are mostly controlled by current so sufficient current would be a good choice. Furthermore, the SMPS connected at the ends of the PV panels was chosen as a Boost, firstly, because the main problem that arises is that usually not enough voltage is supplied by the solar configuration because of unstable and decreased solar irradiance so the voltage needs to be increased and secondly, because the input voltage limit for the Buck SMPS is 8V and there is a possibility this might be reached by the PV panel voltage.

First step for understanding the capabilities of the PV panels and the connection of them to the Boost SMPS was the characterization of the PV panel. For this initial test the SMPS was used just for drawing different currents by changing duty cycle and given resistors were connected as a load to represent an energy dump. The input voltage and current were meant to be measured and not the output ones as the latter represent a characterization of the load ohmic resistor connected which is not the desired test. After testing the input measurements of a PV panel under the constant high intensity sun and using the method of extrapolation to three parallel PV panels, meaning their resistance decreases by three, voltage stays the same and current increases by three, the following IV curves and MPPT had been created.

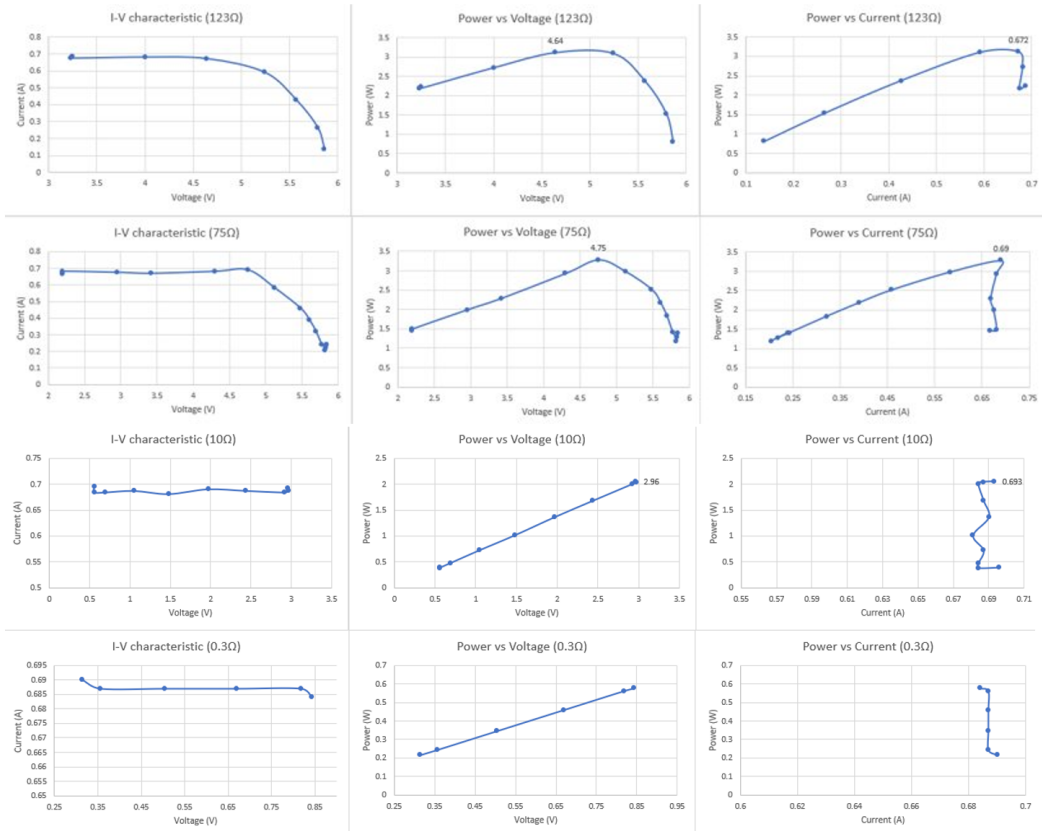


Figure 4.2: Operation curve of PV panel under different operating conditions

The conclusions from the measurement of the PV panels initial test were that for the higher resistances (75 and 123) the input current falls for higher voltages, while for the lower ones (0.3, 2.2, 10) the SMPS holds the input current constant for the various voltages and the V_{in} range is smaller because the current is larger so V_{out} is small. The I-V curve depicts a straight-line region because for low voltages the current in the PV panel stays the same. Then, the next step was to test the LED beacons from a PSU to check measurements at maximum and useful brightness and the results from the test for the yellow, red and blue ones were 2.1V/0.227A, 2.2V/0.130A and 2.9V/0.009A respectively.

The Boost SMPS is programmed with a code that does the following main things. First, ensures that Boost OL non-synchronous operation is applied and if the output voltage is under 17V so in safe operation then MPPT algorithm is allowed to be used. The MPPT algorithm developed is a Perturb and Observe algorithm that compares the current input power with the previous one and if the power has increased then the duty cycle is increased and it continues in the same duty step trend, otherwise duty cycle is decreased and the direction is reversed. After the MPPT function returns the duty cycle then it is saturated between limits before applied to the PWM signal to avoid MOSFET damage and power losses. The following MPPT logic can be seen in the flowchart figure 6.3 below.

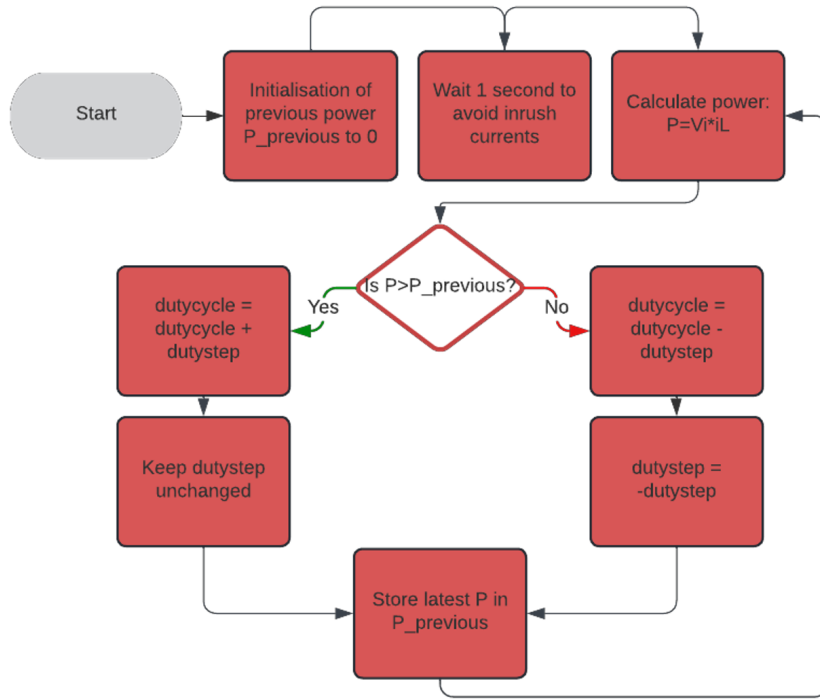


Figure 4.3: MPPT implementation

Another additional delay added and the duty step size were chosen to avoid the sudden drops of the input power to 0 when the inductor current dropped to 0 once DCM was reached.

The LED drivers were coded to first measure the pin voltages in ADC digital 16-bit measurements and then calculate the real number voltages across the devices. Then, it programs them to operate as closed loop Bucks with just a closed loop current PID controller that gives a duty cycle demand based upon the error between the current aim and measured current which runs through the current sensor resistor. Only a current controller was chosen as LEDs are affected more by current unlike resistors. The current aim was set to 0.1A and the current limit to 0.3A which is the rating of the LEDs and the PID values (kpi and kii) were increased after testing so the beacons light up and follow current changes faster.



Figure 4.4: Powering beacons with drivers connected demonstration

Testing was performed on the whole configuration for checking the capabilities of the supercapacitor. The minimum time for the LED to turn on was 4 seconds and the maximum time of available power for the LED even without PSU or PV panels connected or with insignificant current close to 0 was 1.03 minutes.

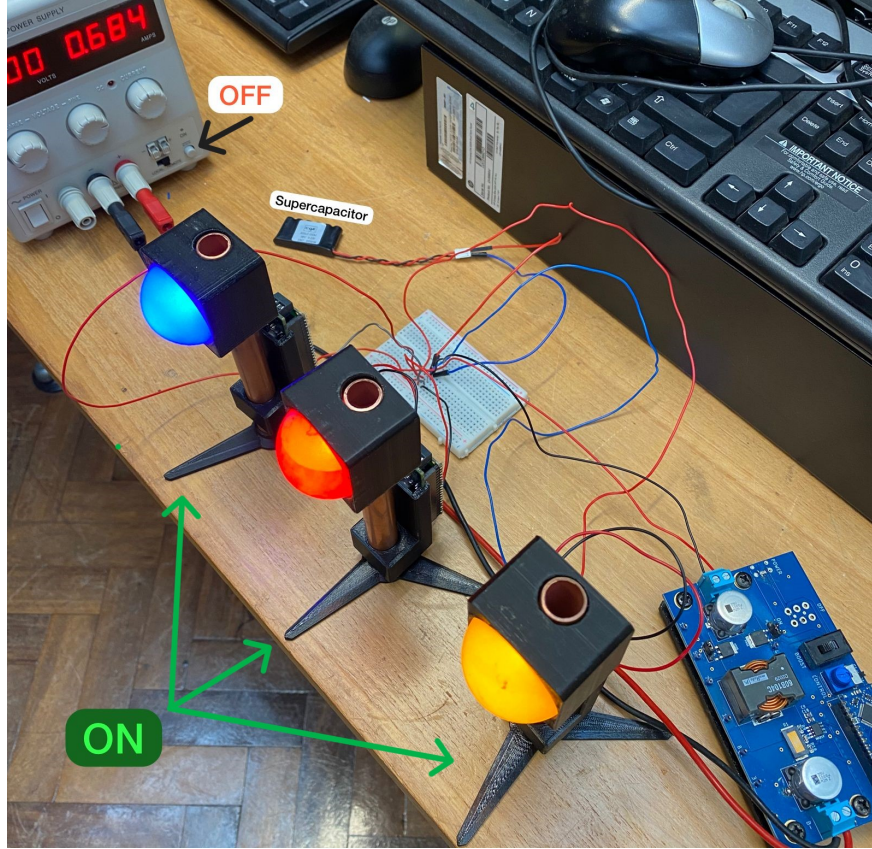


Figure 4.5: Beacons being illuminated despite OFF PSU

4.4 Future development

In the case that the bidirectional SMPS that was intended to control the supercapacitor was available a lot of benefits would appear in contrast to just using the supercapacitor connected in parallel straight to the DC grid. First, energy can be quickly stored and released via the supercapacitor. The supercapacitor may be charged from the grid when there is surplus energy available and discharged back into the grid when there is a need for more power and this is done in an efficient way, as also synchronous operation would be possible. Additionally, to guarantee that the supercapacitor runs within its voltage restrictions, a bidirectional SMPS could regulate the charging and discharging operation. It may control the power flow to avoid overcharging or over-discharging the supercapacitor by monitoring the voltage level and making necessary adjustments. To implement the bidirectional SMPS to interface the supercapacitor, a current closed loop PID control should be used. When energy is needed to the dc grid

then the current will be set to negative so it goes the other way and the supercapacitor is discharged, if the current is set to 0 then no energy passes through the SMPS and therefore to the supercapacitor, if the current is set to positive then the supercapacitor is being charged. If we use Boost then the voltage range is going to be higher, if we use Buck the voltage range will be lower.

Chapter 5

Camera vision

5.1 HSV vs RGB

RGB (red, green, blue) is an additive colour model which mix three primary colours to reproduce other colours.

HSV (hue, saturation, value) is a color model that remaps the RGB primary colours into a different dimension in the human-familiar form. The hue value normally ranges from 0° to 360° with red primary at 0° or 360° , green primary at 120° and blue primary at 240° . Both saturation and value are represented in the percentage format. Saturation shows the degree of gray and value describes the intensity or the brightness. In order to fit the HSV values, a re-scale was required to convert all of them into the range of 0 to 255. The conversion is in figure 5.1. The overview of the HSV model is shown in Figure 5.2.

```
always @(*) begin
    max_val = (red > green) ? ((red > blue) ? red : blue) : ((green > blue) ? green : blue);
    min_val = (red < green) ? ((red < blue) ? red : blue) : ((green < blue) ? green : blue);
    delta = max_val - min_val;

    // Calculate hue component
    if (delta == 0) begin
        hue = 0;
    end else if (max_val == red) begin
        hue = ((green < blue) ? (green - blue + 256) : (green - blue)) * 43 / delta;
    end else if (max_val == green) begin
        hue = ((blue < red) ? (blue - red + 256) : (blue - red)) * 43 / delta + 85;
    end else begin
        hue = ((red < green) ? (red - green + 256) : (red - green)) * 43 / delta + 171;
    end

    // Calculate saturation component
    if (max_val == 0) begin
        saturation = 0;
    end else begin
        saturation = (delta * 255) / max_val;
    end

    // Calculate value component
    value = max_val;
end
```

Figure 5.1: conversion from RGB to HSV

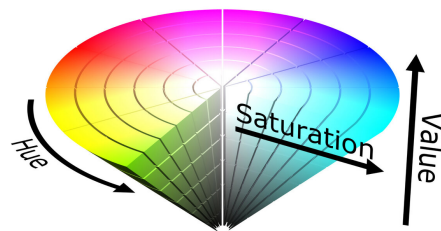


Figure 5.2: HSV model

5.1.1 HSV Advantages

1. HSV model is more robust to the high brightness images. The actual beacons are too bright and it is hard to switch the threshold values with the RGB model. However, this issue is solved by only tuning the value in HSV.

2. HSV provides a more intuitive representation of colours. Characteristics like dominant colour, purity and brightness are separated and determined by three individual values in HSV which are mixed in the RGB model.

5.1.2 HSV Limitations

1. HSV has a limited colour range, which means some dark colours cannot be accurately represented by the HSV model because of its colour distribution.
2. HSV tends to have more noises due to colour mixing in white lines. The white light is the combination of primary colours and it is difficult to define the range of hue due to the unclear boundary between colours within the model.

5.2 Filtering

Filtering was especially important when we get positions of beacons, for example, the red beacon appears to have yellow features when the led is too bright and there are often some background random noises of the light being detected. The original bounding box was too sensitive to these noises and did not have a high accuracy when outputting the coordinates of beacons.

5.2.1 1-Dimensional Filter

A counter was used to count the numbers of different color pixels horizontally every 16 pixels. In the first situation, where the beacons were far away from the camera, the majority of the pixels were gray, and the desired color was to be found in the remaining minority, as shown in the first row of Figure 5.3. In the second situation, when the beacons were close enough, most pixels within every 16-pixel segment were classified as the same color, as shown in the second row of Figure 5.3. All other situations were considered random noises.

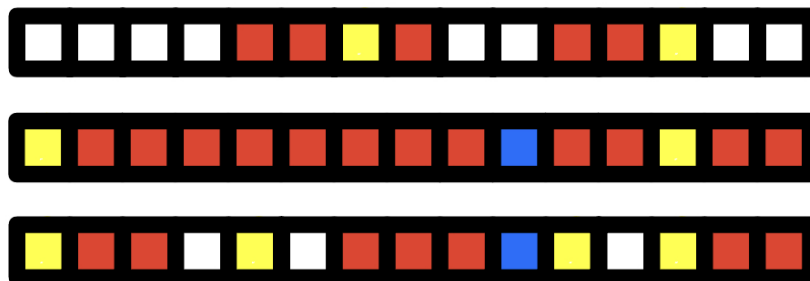


Figure 5.3: Possible Colour Distribution

To implement this counter, we need $4(\text{colours}) \times 16(\text{bits for each}) = 64$ registers which does not take a large space in the FPGA.

This method had drawbacks of low accuracy when recognizing the colour. The image was evenly divided into $640 \div 16 = 40$ different columns, which meant there were only 40 different coordinates instead of 640 when updating the positions of the beacons. And also this method did not consider vertical situations, the vertical coordinates were still affected by the noises.

5.2.2 2-Dimensional Filter

The most ideal way of removing noises in the image were by using Gaussian Filters which convolve the image with a Gaussian kernel to produce a more smoothing image. This kernel takes the values of the target with its surrounding pixels and allocates different weightings to each of them. shown in figure 5.4 and figure 5.5.

$$\frac{1}{16}$$

1	2	1
2	4	2
1	2	1

Figure 5.4: 3×3 Gaussian kernel

$$\frac{1}{273}$$

1	4	7	4	1
4	16	26	16	4
7	26	41	26	7
4	16	26	16	4
1	4	7	4	1

Figure 5.5: 5×5 Gaussian kernel

However, this method had limitations when we apply it to the FPGA. The FPGA had a finite amount of space and the way of processing image was pixel by pixel from top-left to bottom-right, which meant we need a large space to store the information of surrounding pixels. For example, the current resolution is 640×480 , so a $3(\text{RGB}) \times 8(\text{bits per colour}) \times 3(\text{rows}) \times 640(\text{number of pixels each row}) = 46080$ registers were needed and it was not realistic to implement.

5.3 Method of Detection

RGB was utilized for detecting white lines, while HSV was used for beacons. HSV proved to be more effective in distinguishing the boundaries of beacons, but it also introduced unwanted noise. This was not ideal for detecting white lines, as it increased the computational burden and led to unstable bounding box outputs.

Figure 5.6 illustrates the threshold values utilized in the detection process. The colors

of beacons were mainly determined by the hue value, while the remaining values were relatively similar. Due to the limitations of HSV in detecting mixed colors, RGB was employed instead for white line detection.[2]

```
assign red_detect = (hue <= 8'd13) & (saturation > 8'd120) & (value > 8'd70);
assign yellow_detect = (hue > 8'd13) & (hue < 8'd35) & (saturation > 8'd110) & (value > 8'd50);
assign green_detect = (hue > 8'd140) & (hue < 8'd240) & (saturation > 8'd90) & (value > 8'd50);
assign white_detect = (red > 8'd30 & red < 8'd150) & (green > 8'd40 & green < 8'd210) & (blue > 8'd30 & blue < 8'd110);
```

Figure 5.6: Threshold Values Used in Detection

5.3.1 For White Lines

The state machine in Figure 5.7 was utilized to distinguish the boundaries of white lines with value '0' represents non-white pixels, while '1' represents white pixels. The following processes are employed:

1. If the previous pixel was 0 and the current pixel is white, the value is set to 1, indicating that the current pixel is at the boundary of a white line.
2. If the previous pixel was 1 and the current pixel is white, the registers remain unchanged, indicating that the pixel is inside the white line.
3. When the current pixel is not white, the register is always reset to 0, indicating that it is not inside the white line.

These processes were repeated both directions, requiring 640 registers horizontally and 480 registers vertically. The raw image of the white lines is shown in Figure 5.8, while Figure 5.9 displays the corresponding image after undergoing the described processing.

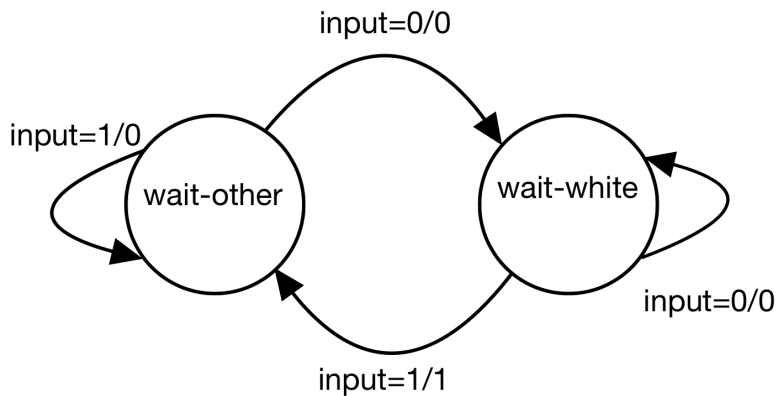


Figure 5.7: State Machine for White Line Detection

This method had the disadvantage that large number of the points were required for each image, which took longer time for the ESP32 to receive and send all coordinates for one image.



Figure 5.8: Raw Image before Processing



Figure 5.9: Image after Processing

This problem could be solved by decreasing the sampling rate of white lines. The detection process only happened each 16 pixels, so the total number of white points needed to be send was reduce to 1/16 for each image. The original method required at most 6000 points to send in the worst case and it was reduced to 400 points for the same situation.

5.3.2 For Beacons

Bounding boxes were used for beacon detection and each of them consists of four lines indicating upper and lower limit for both horizontal and vertical direction respectively. By only sending the data of the top-left and bottom-right was enough to fully get information of each beacon. Figure 5.10 and figure 5.11 show the bounding boxes.

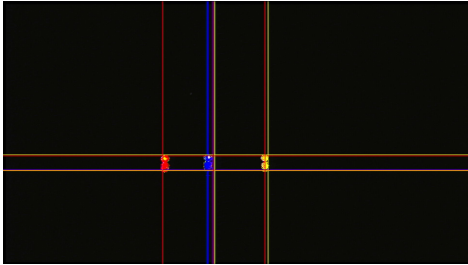


Figure 5.10: Beacons in Far Distance

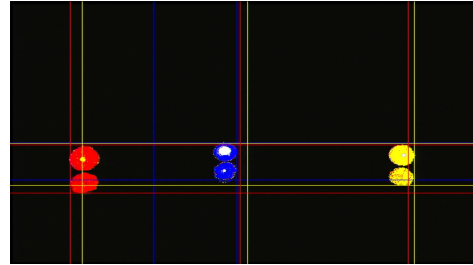


Figure 5.11: Beacons in Close Range

5.4 Transmitting with ESP32

A state machine was used in FPGA that followed the sequence in Figure 5.12 to transmit data with the ESP32. The transmission process involved the following steps:

1. The initial data transmitted was always "END," which indicated the start of a new transmission loop.
2. Next, "RBB" was transmitted, followed by the coordinates of the red bounding box, specifying the top-left and bottom-right positions.

3. The same process was repeated twice more: "YBB" for yellow and "GBB" for blue, transmitting their respective bounding box coordinates.
4. Subsequently, the states "WBB" and "White-Coordinate" were sent. The state "White-Coordinate" was repeated up to a maximum of 33 times, implying that at most 33 points were sent in each iteration.

This sequence allowed the FPGA to transmit data in a structured order, ensuring that the necessary information regarding bounding boxes and white coordinates was sent accurately.

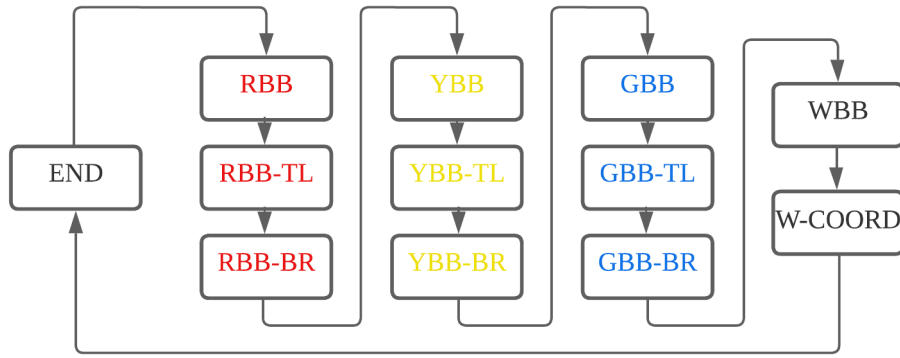


Figure 5.12: State Machine for Data Transmission

A 128×32 -bit array was utilized to store the coordinates of white points. These coordinates were stored in the array after all the previous data in it have been send. Once the array was fully populated with 128 points, a state machine initiates the transmission of all the data in this array. Two 11-bit registers were employed to store and track the last transmitted data from the array. During subsequent transmissions, only the points with coordinates larger than those stored in the registers need to be stored in the array. As soon as the tracked coordinates reach a sufficiently high value, the state machine ceased the transmission of white coordinates, indicating the completion of one image.

Because in the worst case, there were approximately 400 points for each image, the above process needed to repeat around $400/33 \approx 13$ times. By considering there might be situations where 400 coordinates were not enough for one image, registers for tracking the last send point were reset after sending 16 times or 528 points. The camera had the image refresh rate of 60 frames per second and the interval of sending messages was 20 frames, so it took $16 \times 20/60 \approx 5.33$ seconds to send one image.

Chapter 6

Communication

6.1 Communication between FPGA and ESP32

Communication between the FPGA and ESP is established using the UART protocol, due to its simplicity and efficiency, although SPI was implemented in the initial design, by designing and wiring two Verilog modules to ensure communication between Master (FPGA) and Slave (ESP).[3] However, this was not used in the final design, due to UART proving adequate, while at the same utilizing minimum resources, two GPIO's as opposed to four.

The buffer size of the UART is 8 bits, with the data arriving in reverse order (LSB to MSB). This means that in order to reconstruct the 32bit message, 4 transactions are needed for each word, as well as a reversion of the received sequence. The code for handling the data, resembles a state machine, which follows the same format described in Figure 5.12 The ESP32 code runs in one core, receives the data, reconstructs the message with appropriate byte manipulation and performs error detection and correction, to extract the useful data and format it along with some additional information to be sent to the SQL server.

The high amount of data transmitted in each message, caused multiple packets to be corrupted upon the receipt by the ESP. This led to the implementation of multiple error-correcting methods, including a State Machine format for receiving the headers of the beacons data, as discussed above. Finally, in case of Packet Loss, incorporating the value 1 in the most significant bit (MSB) of the corrupted Data, introduced a distinctive indication of an error in transmission, as it would grow out of bounds and be disregarded by the server, since the correct data are composed of a 0 in the MSB. Parallel communication was also considered, through the sending data over separate UART transmission lines, however, was rendered as redundant, since the error-detection and correction allowed the reduction of package Loss, with adequate data being received and plotted by the Server.

6.2 Communication between ESP32 and Server

Regarding the data to be transmitted, the coordinates of the beacons and the white lines are stored in an array, with an “BeaconType” indicator, for the Server to be able to distinguish what each coordinate represents. In addition, an NTP Server is initiated in order to acquire a Timestamp in ISO format, to ensure in order data-transfer, as well as be utilized when querying the database. A Json containing all the appropriate data is initiated and sent to the Backend Server, in an appropriate format to be stored in the database.

Establishing communication between the ESP and the Server was accomplished through the use of WebSockets, as opposed to traditional HTTP Requests. The preliminary execution, of sending Packages to the Server, was established through a function, that after each call would establish a new connection to the server and execute a POST request. This would cause delay, both in reading the Data from the ESP, but also in sending it to the Server to be stored in the database. In the first case, delay in reading the data would cause incoming packages to become dropped/lost, as we do not read them while we wait for the HTTP request to be complete. In the second case the delay would extend the time needed to acquire each frame. An optimization for this was implemented, by taking advantage of the Dual Core ESP, using the first Core to read the data and the second to send them using HTTP Post request. However, this would require utilizing both cores of the ESP which is not possible since one of the cores must be used by control.

Therefore, the use of WebSockets was deemed most suitable. WebSockets allow real-time communication with the server. Unlike traditional HTTP requests, WebSockets establish a persistent connection with the server, eliminating the need for repeated handshakes (overhead) and allowing instant transmission of data.

6.3 Communication between Control and Vision ESP32s

The motor control uses microstepping, meaning that the signal has to be send to the motor at a frequency high enough to ensure a smooth operation. However, the data receiving process at each iteration of the control loop will drastically reduce loop speed, making the motor unable to turn as fast, leading to oscillation, noise and overall an unstable system. Since, the Control ESP device utilizes both cores for its operations. The challenge it faces a is caused by the delays due to wifi connectivity. To resolve this issue, it is necessary to isolate the slow part of the wifi processing outside the control loop. This allows the control signals to be processed efficiently in the Control ESP, while the remaining core of the Vision ESP focuses on translating the wifi signals to SPI signals. This approach allows for optimized performance of wifi functionality into the Control ESP system. Finally, for the transmission of data from the Vision ESP to the Control ESP,the I2C Protocol is implemented, with the Control ESP acting as

the Primary and the Vision ESP as the Secondary device. Through the use of the I2C Protocol, the design can use a Primary-Secondary architecture, allow serial communication between the two devices and achieve reliable data transfer.

6.4 Conclusion

The ESP Microcontrollers caused multiple issues during integration that required an extended amount of time to be resolved. After multiple testing and adjustment, the design choices seemed to prove the most appropriate amongst the other methods considered. The end result of this implementation, showed that the System was able to perform at a very high pace, allowing rapid response to sudden switches of the Camera, accounting for possible imbalance of the Rover at certain moments. To conclude, through the combined power of both ESPs an optimized model with improved efficiency has been achieved.

Chapter 7

Data processing

7.1 Initial Setup

Our database was deployed on an Amazon EC2 (Elastic Compute Cloud) instance, eliminating the need to invest in the hardware up front. T2.micro was selected for our application. This decision was mainly guided by its cost-effectiveness and suitability for applications with moderate compute requirements.

Ubuntu 22.04 LTS was chosen as our operating system due to its wide-ranging support and compatibility with various software. For the database management system, MariaDB was selected due to its scalability and ease of use. MariaDB is a fork of MySQL and is fully compatible with MySQL.

7.2 Database Design

Our database was named ‘Testdb’. There was one table Points in it. The table schema is shown in the figure 7.1.

Field	Type	Null	Key	Default	Extra
id	bigint(20)	NO	PRI	NULL	auto_increment
timestamp	datetime	YES		NULL	
BeaconType	varchar(255)	YES		NULL	
xcoordinate	int(11)	YES		NULL	
ycoordinate	int(11)	YES		NULL	

Figure 7.1: Table Description

The primary key in this table was id, which was automatically assigned and incremented when each entry was inserted. It had a type of bigint(20) and should be sufficient in our application. Possible values for BeaconType include:

- ‘ColourTL’: The top left corner of a color bounding box, colours include red, yellow and blue;

- ‘ColourBR’: The bottom right corner of a colour bounding box;
- ‘WL’: A point on a white line.

The coordinates of a point are formed of x and y coordinates. These were stored in decimal. The possible ranges of valid xcoordinate and ycoordinate values were:

- $0 \leq xcoordinate \leq 640$;
- $0 \leq ycoordinate \leq 480$.

7.3 Backend Overview

This section of the report focuses on the setup of our backend server, which interfaces with our MariaDB database and serves data to our frontend. This server was also hosted on AWS for consistent performance and availability.

7.4 Server Configuration

Our backend server was built with Node.js and Express, with several middleware libraries to extend its functionalities. These were implemented:

- **CORS:** To handle “Cross-Origin Resource Sharing” and make sure the server can process requests from different domains.
- **WebSocket:** In the earlier stages of testing, HTTP was used for data transmission, but the latency performance was not satisfactory. WebSocket was hence deployed to facilitate real-time bidirectional communication between the server and the client, in the hope of reducing latency.
- **mysql:** To connect and interact with our MariaDB database.

The server listened on port number 3001 by default.

7.5 Database Connection

Our server was connected to the MariaDB database using the mysql library. A connection object (con) was created using the mysql.createConnection method. The parameters provided within the object were:

- **host:** the IP address of the host where the database is located.
- **user:** the username used for authentication with the database.
- **password:** the password associated with the aforementioned username for authentication purposes.

- **database:** the specific database on the host that the connection should be made to.

After the creation of the connection object, the `con.connect` method was called to initiate the connection. If no error occurred, a message stating "Successfully connected to the database..." was logged to the console.

7.6 Data Manipulation

`Express.json()` was used to parse incoming requests with JSON payloads and `express.static()` was used to serve static files. The CORS middleware was configured to allow all origins (*) and various HTTP methods, simplifying testing by removing the need to consider IPs when testing.

An API endpoint 'POST /api/data' that accepted an array of records in the HTTP request body and inserted them into the Points table was created. A Date object for the incoming timestamp was created and the timestamp was then formatted by using:

- `.toISOString()`: converts a Date object to a string using the ISO 8601 extended format. After this step the string would be in the the same format as this example: **2023-06-19T13:34:55.123Z**.
- `.slice(0, 19)`: extracts the first 19 characters of the string, removing the milliseconds and the timezone indicator.
- `.replace('T', ' ')`: This replaces the 'T' separator between the date and time with a space. The final output would be: **2023-06-19 13:34:55**. The string is then ready to be inserted into the database.

For testing purposes, an API endpoint 'GET /pointQuery' was also created to return all records from the 'Points' table in the 'Testdb' database. The flow chart for this part is shown in fig 7.2. The query and the query result are shown in fig 7.3 and fig 7.4.

7.7 Frontend Overview

The frontend of the application was implemented using Vue.js, a modern JavaScript framework for building user interfaces. During the testing stage, data was fetched from the database via the backend server, and the points were rendered accordingly.

7.8 Application Layout

`App.vue` served as the main layout of the application. There were three key components:

- `MazeTitle`

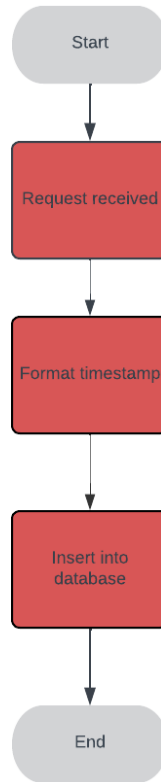


Figure 7.2: Data flow diagram

- MazeFrame
- ControlButtons

The pictures for the frontend dot plotting and the corresponding camera view are shown in fig 7.5 and 7.6.

7.9 VUE Components

MazeTitle was responsible for displaying the title of the website, which was “The Minotaur’s Thread Rover”. The title was center-aligned, had a size of 3em and a bottom margin of 20px.

MazeFrame was the main display area. It had a width of 640 px and a height of 480 px. During testing, it was mainly responsible for drawing white lines represented by the points and marking the position of the rover with ‘X’.

```

app.get("/pointQuery", (req,res)=>{
  con.query("SELECT * FROM Points", function (err,result,fields){
    if(err) throw err;
    res.json(result)
  });
});

```

Figure 7.3: Code for query

Figure 7.4: Query result

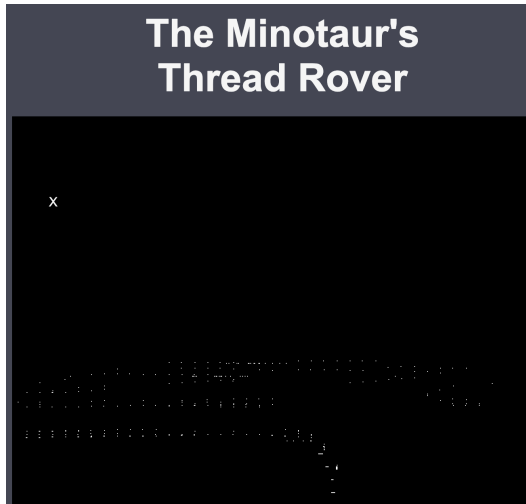


Figure 7.5: Example of the website layout when plotting test data.

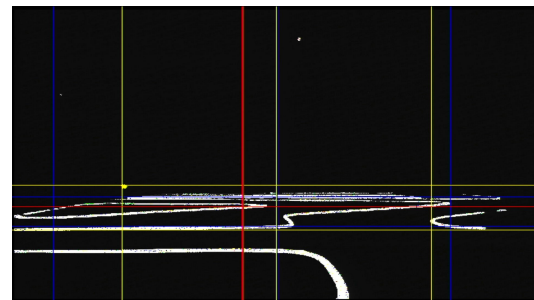


Figure 7.6: The camera view

7.10 Maze Frame Details

7.10.1 Template Section

At the testing stage, this section was responsible for drawing the points and the rover. It took each point in the array points and drew the required pattern according to their x and y values. There were separate classes for 'point' and 'rover' so they were rendered separately. At the production stage a most recent map of the maze overlaid with the rover position and a shortest path found would be plotted in the frame.

7.10.2 Script Section

Most of the logic and operations was contained in this section. There were in total five methods in this section.

- fetchData()
- getLatestEntries()
- averageCoordinates()

- startPolling()

A diagrammatic representation of the logic at the testing stage can be seen in fig 7.7.

7.10.3 Style Section

The points were defined as 1px by 1px squares. The position was set to absolute and the color was set to #FFF (white). The rover had its position set to absolute and was represented by 'X'.

7.11 Older Version

The older version of this program took in a fixed string and processed it. This version was developed to test FPGA output when the ESP side was not ready. A coordString was taken in and '—' was removed. And the string was separated to obtain point coordinates. It also zeroed out the five most significant so only the bottom 11 bit was retained. An example of the coordString was: 00574242|01620134|01630131.

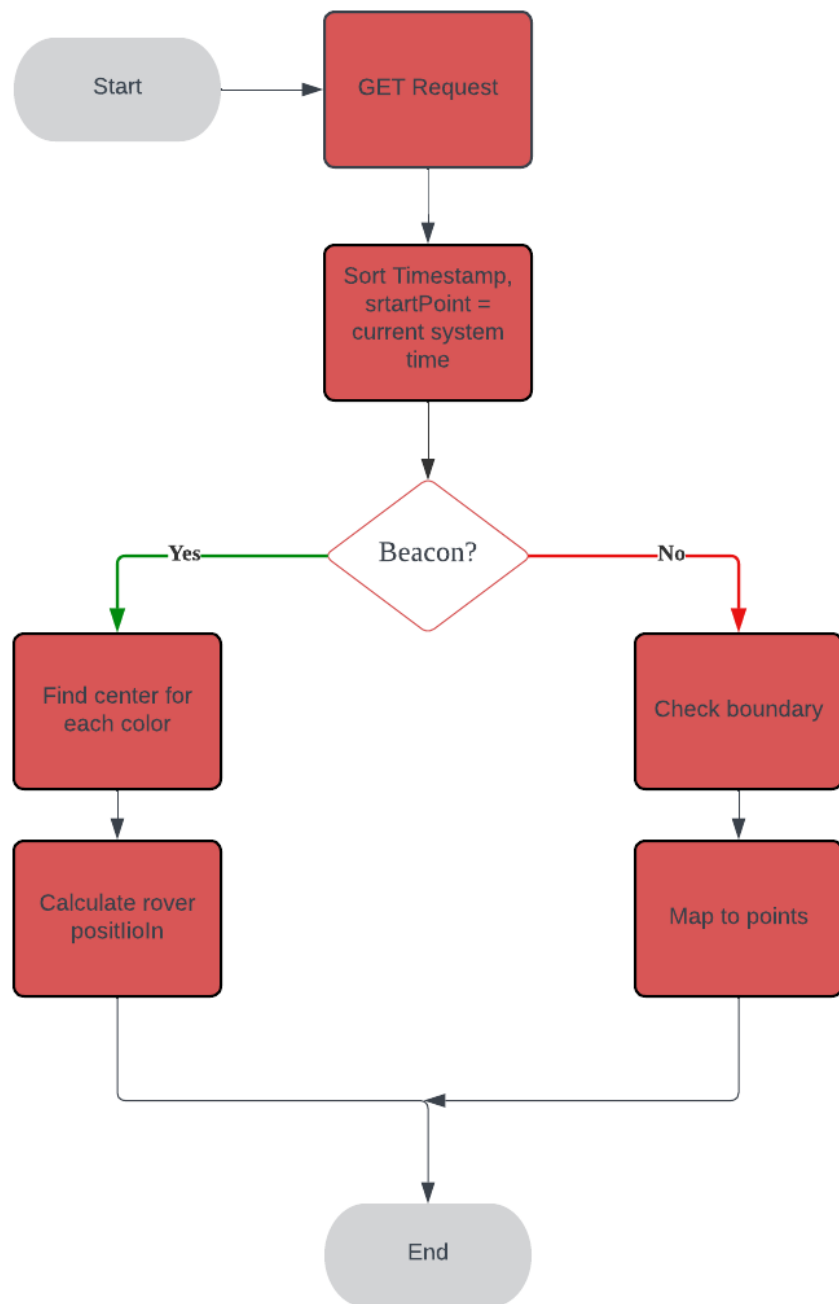


Figure 7.7: Frontend flow diagram

Chapter 8

Navigation / Backend Algorithm

8.1 Positioning

8.1.1 Overview

The goal of positioning is to use the three beacons and the camera to calculate the horizontal and vertical position of the robot. The known variables are the distance between the three beacons, the angle between the beacons, the position of the three beacons, and the angles between any two of the beacons and the camera.

The positioning is done based on the relative positions(angles) of the beacons on camera's field of view. The setup can be simplified as a 4 dot system. Three of the dots are the position of the beacons and one of the dots is the camera on the robot, as the drawing in figure 8.1.

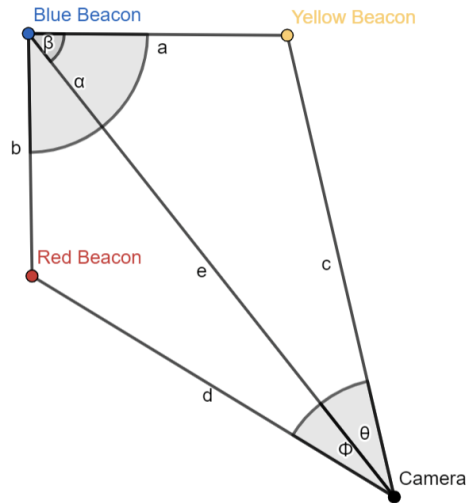


Figure 8.1: Position Finding Algorithm Geometry Model, where α , a , and b are known parameters and ϕ and θ are variables measured by camera

Through trigonometry, the distance between the robot and the beacons can be calculated. A generic equation deduced is in equation 8.1. Another equation is also

used on the actual code implementation that involves calculating β as in equation 8.2. The deduction can be seen in appendix.

$$c = \frac{ab \sin(\phi + \alpha) - a^2 \sin(\phi)}{\sqrt{a^2 \sin^2(\phi) + b^2 \sin^2(\theta) + 2ab \sin(\theta) \sin(\phi) \cos(\alpha + \phi - \theta)}} \quad (8.1)$$

$$\beta = \arctan\left(\frac{b \sin(\theta) \sin(\alpha + \phi) - a \sin(\phi) \sin(\theta)}{b \sin(\theta) \cos(\alpha + \phi) + a \sin(\phi) \cos(\theta)}\right) \quad (8.2)$$

8.1.2 Code Implementation

The generic equation of c (equation 8.1) is elegant but relatively inflexible. For example, it would require recalculating angles such as β which it was done within this equation. The code implementation introduces equation 8.2 in order to calculate other variables.

The workflow is as figure 8.2. When activated, the positioning routing keeps querying the database for the beacon data from the camera. After getting the data, the comparer returns the x coordinates of middle, left, and right beacons. With these coordinates, the `getAngle()` function can calculate ϕ and θ in figure 8.1. Using equation 8.1 and equation 8.2, `angle2distance()` can calculate β and c . They are then feed into `getXY()` to calculate the x and y coordinates of the robot relative to the beacons.

Comparer

Comparer returns the left beacon position, middle beacon position, and right beacon position. `red_x_min`, `yellow_x_min`, `red_x_max`, `yellow_x_max`, `blue_x_min`, `blue_x_mid`, and `blue_x_max` is fed into the comparer. From the experiment, it is known that the blue beacon would not be misidentified as other beacons. Thus, blue beacon is set as the centre beacon. To get the position of the left beacon, `red_x_min`, `yellow_x_min`, `red_x_max`, and `yellow_x_max` are selected to leave only the ones with coordinates that are less than `blue_x_mid`. Then they are compared with each other to leave the one with the least x coordinate value. Similar operation is done with the position of right beacon.

`getAngle()` Function

`getAngle()` converts the position on camera into angles. In short, it is a normalization from difference in position into angles between beacons and camera. For ϕ , equation 8.3 is used. For θ , equation 8.4 is used.

$$\phi = (x_{mid} - x_{left}) \cdot \frac{\text{Field of View}}{x_{border}} \quad (8.3)$$

$$\theta = (x_{right} - x_{mid}) \cdot \frac{\text{Field of View}}{x_{border}} \quad (8.4)$$

$x_{right}, x_{mid}, x_{left}$ are the x coordinates of the right beacon, middle beacon, and left beacon respectively. x_{border} is the x resolution, which is 640. Field of View is $70^\circ / 1.22173$ radians according to camera datasheet.[4]

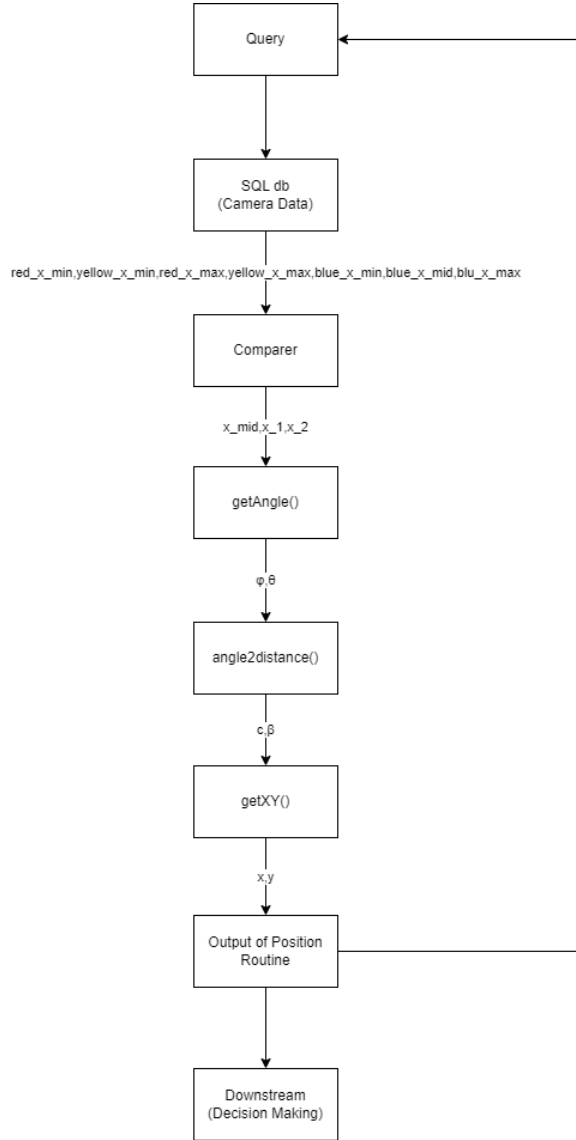


Figure 8.2: Workflow of Position Finding Routine Algorithm

angle2distance() Function

angle2distance() is the code implementation of equation 8.1 and equation 8.2. It takes in ϕ and θ calculated from getAngle(), a and b set when placing down beacons, and α which is assumed to be $90^\circ / 1.5708$ radians.

getXY() Function

getXY() Function converts the calculated value of c and β into x coordinate and y coordinate of the robot. According to figure 8.3, γ is calculated by $1.5708 - \beta - \theta$, x is calculated by $\sin(\gamma) \cdot c + a$, y is calculated by $\cos(\gamma) \cdot c$.

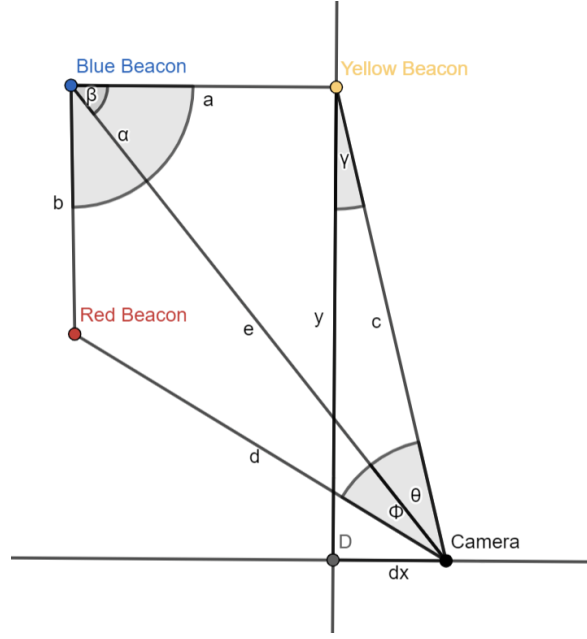


Figure 8.3: Calculating X and Y Coordinates of Robot

8.2 Mapping

8.2.1 Transformation

The core component of the mapping algorithm is the transformation of the view of the camera. As the camera is viewing the field at a right angle, the white lines conducted is not the actual dimension of the field. A perspective transformation is thus required to transform the angled view into the top view as shown in figure 8.4.

Using the perspective warp matrix as equation 8.5 [5], it is able to warp all the points. As the camera has a constant viewing angle, the perspective warp matrix is constant as well.

$$\begin{bmatrix} y_w \cdot y'_u \\ y_w \cdot y'_v \\ y_w \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & t_u \\ a_{21} & a_{22} & t_v \\ p_0 & p_1 & p \end{bmatrix} \begin{bmatrix} x_u \\ x_v \\ 1 \end{bmatrix} \quad (8.5)$$

where y'_u and y'_v are the transformed coordinates of the points x_u and x_v

The calculation of the transformation matrix is solving the set of equations shown in equation 8.6.

$$\begin{bmatrix}
x_1 & y_1 & 1 & 0 & 0 & 0 & -x_1 \frac{h_{11}x_1+h_{12}y_1+h_{13}}{h_{31}x_1+h_{32}y_1+h_{13}} & -y_1 \frac{h_{11}x_1+h_{12}y_1+h_{13}}{h_{31}x_1+h_{32}y_1+h_{13}} \\
x_2 & y_2 & 1 & 0 & 0 & 0 & -x_2 \frac{h_{11}x_2+h_{12}y_2+h_{13}}{h_{31}x_2+h_{32}y_2+h_{13}} & -y_2 \frac{h_{11}x_2+h_{12}y_2+h_{13}}{h_{31}x_2+h_{32}y_2+h_{13}} \\
x_3 & y_3 & 1 & 0 & 0 & 0 & -x_3 \frac{h_{11}x_3+h_{12}y_3+h_{13}}{h_{31}x_3+h_{32}y_3+h_{13}} & -y_3 \frac{h_{11}x_3+h_{12}y_3+h_{13}}{h_{31}x_3+h_{32}y_3+h_{13}} \\
x_4 & y_4 & 1 & 0 & 0 & 0 & -x_4 \frac{h_{11}x_4+h_{12}y_4+h_{13}}{h_{31}x_4+h_{32}y_4+h_{13}} & -y_4 \frac{h_{11}x_4+h_{12}y_4+h_{13}}{h_{31}x_4+h_{32}y_4+h_{13}} \\
0 & 0 & 0 & x_1 & y_1 & 1 & -x_1 \frac{h_{21}x_1+h_{22}y_1+h_{23}}{h_{31}x_1+h_{32}y_1+h_{13}} & -y_1 \frac{h_{21}x_1+h_{22}y_1+h_{23}}{h_{31}x_1+h_{32}y_1+h_{13}} \\
0 & 0 & 0 & x_2 & y_2 & 1 & -x_2 \frac{h_{21}x_2+h_{22}y_2+h_{23}}{h_{31}x_2+h_{32}y_2+h_{13}} & -y_2 \frac{h_{21}x_2+h_{22}y_2+h_{23}}{h_{31}x_2+h_{32}y_2+h_{13}} \\
0 & 0 & 0 & x_3 & y_3 & 1 & -x_3 \frac{h_{21}x_3+h_{22}y_3+h_{23}}{h_{31}x_3+h_{32}y_3+h_{13}} & -y_3 \frac{h_{21}x_3+h_{22}y_3+h_{23}}{h_{31}x_3+h_{32}y_3+h_{13}} \\
0 & 0 & 0 & x_4 & y_4 & 1 & -x_4 \frac{h_{21}x_4+h_{22}y_4+h_{23}}{h_{31}x_4+h_{32}y_4+h_{13}} & -y_4 \frac{h_{21}x_4+h_{22}y_4+h_{23}}{h_{31}x_4+h_{32}y_4+h_{13}}
\end{bmatrix} \times \begin{bmatrix} h_{11} \\ h_{12} \\ h_{13} \\ h_{21} \\ h_{22} \\ h_{23} \\ h_{31} \\ h_{32} \end{bmatrix} = \begin{bmatrix} \frac{h_{11}x_1+h_{12}y_1+h_{13}}{h_{31}x_1+h_{32}y_1+h_{13}} \\ \frac{h_{11}x_2+h_{12}y_2+h_{13}}{h_{31}x_2+h_{32}y_2+h_{13}} \\ \frac{h_{11}x_3+h_{12}y_3+h_{13}}{h_{31}x_3+h_{32}y_3+h_{13}} \\ \frac{h_{11}x_4+h_{12}y_4+h_{13}}{h_{31}x_4+h_{32}y_4+h_{13}} \\ \frac{h_{21}x_1+h_{22}y_1+h_{23}}{h_{31}x_1+h_{32}y_1+h_{13}} \\ \frac{h_{21}x_2+h_{22}y_2+h_{23}}{h_{31}x_2+h_{32}y_2+h_{13}} \\ \frac{h_{21}x_3+h_{22}y_3+h_{23}}{h_{31}x_3+h_{32}y_3+h_{13}} \\ \frac{h_{21}x_4+h_{22}y_4+h_{23}}{h_{31}x_4+h_{32}y_4+h_{13}}
\end{bmatrix} \quad (8.6)$$

From OpenCV there is a function that does this in an efficient way.

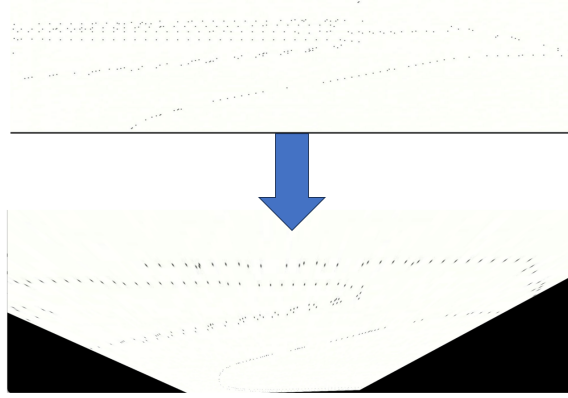


Figure 8.4: A set of points before and after warping

8.2.2 Mapping Routine

The goal for the mapping routine is to map the surroundings of the robot, i.e. all 4 aspects of the robot. Since the camera can only map one side each time, thus the routine consists mapping all 4 aspects of the robot as shown in figure 8.5.

A canvas is set up every time the robot starts. The canvas is the size of the field where the future points and connections are made every routine. Eventually, it will be filled with the obstacles of the maze.

On call, the system will turn 4 times in total, each time query the white lines and apply transformation onto it in order to warp the perspective. The resulting points are then plotted onto the canvas, awaiting for future post-processing.

8.2.3 Mapping Post-processing

To make downstream algorithms understand the orientation of the matrix, it is not enough to have only points. The post-processing program clusters the points that are close to each other (within 20 pixels) as a line. It leaves the blank as routes that the robot can take in the decision making process.

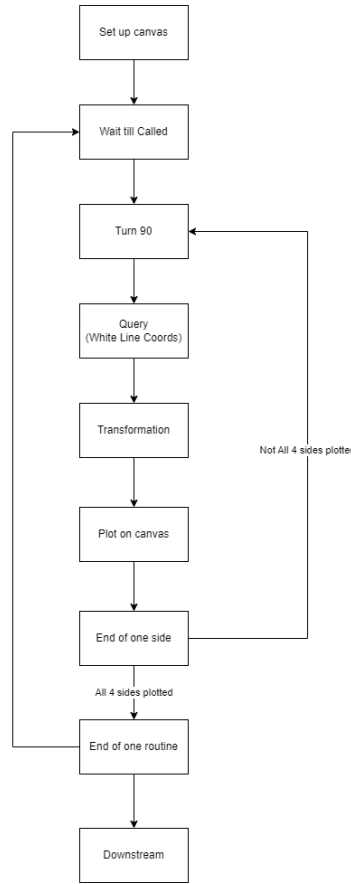


Figure 8.5: Workflow of Mapping Routine Algorithm

8.3 Decision Making

8.3.1 Overview

There are many maze solving algorithms already existed, such as Chain Algorithm, Recursive Backtracker, or Trémaux's Algorithm.[6] However, the goal for the maze robot is not solve the maze but map the maze, where the solution is processed elsewhere. Therefore, the problem is to find a way to map as much field as possible. The algorithm for solving this problem is defined as decision making algorithm.

There are two iterations for the decision making algorithm. The first iteration considered using reinforcement learning, as a leaping goal, which eventually got abandoned due to the lack of time. The second iteration uses a simple greedy decision making routine incorporated with law of large numbers (LLN). The second iteration is the final algorithm adopted.

8.3.2 Second Iteration

The process of mapping is split into short-time-interval steps of routines. Each routine is consisted of the following components as shown in figure 8.6. During each routine, the system first locates the robot by turning to beacons and running the positioning

routine. Then it maps the four sides using mapping routine. After the mapping post-processing is done, the system will have the knowledge on whether routes exist, how many routes exist, and where the routes are. Function `choice()` will be then called and make a choice between routes to try to map the whole field. After the route is chosen and the robot is turned. It will move forward a constant small increment of distance.

The positioning routine, mapping routine, and mapping post-processing are mentioned before in section 8.1 and section 8.2. The core component of this LLN incorporated greedy algorithm is the function `choice()`.

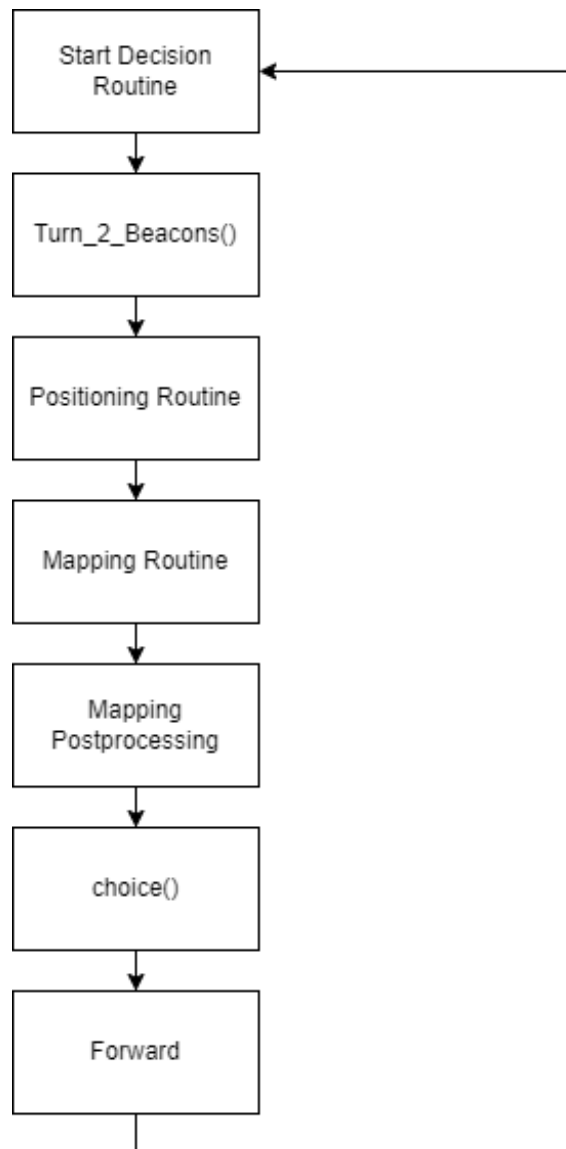


Figure 8.6: Workflow of Decision Routine Algorithm (Second Iteration)

Turn_2_Beacons() Function

The goal of this function is to find the position of beacons by turning. Each time the robot is instructed to turn 10 degrees. The system will then inspect the output of the camera by querying the database to get newest data. If there is no existence of 3 beacons, the system will instruct the robot to turn again. This process will be repeated until 3 beacons are seen.

choice() Function

This function makes a choice on which route to take when there are multiple routes present and gives the turning instruction. If there is only one route, it will choose that route. If there is no route, it will go back until it meets choices. The choice is decided by a weighted randomization on two decision making sub-functions.

The first sub-function is the exploration function. It randomly chooses between the routes with equal weights. It will then calculate the bias angle from the position of the robot and the route. Eventually, the angle is sent to robot to turn.

The second sub-function is the exploitation function. It will randomly choose between the routes that will lead to an area that has not been mapped before. It will then calculate the bias angle from the position of the robot and the route. Eventually, the angle is sent to robot to turn.

The weight of exploration function is 0.2 and the weight of exploitation function is 0.8. This requires further adjustments.

8.3.3 First Iteration

The first iteration considered a simple reinforcement learning model. It tries to maximize the reward which is defined as the new content seen by the mapping algorithm that is not on the part of canvas that contains points. It is abandoned due to the inability to test and train.

Chapter 9

Conclusions

In conclusion, this project has achieved great success in addressing the challenges of balancing the rover vertically on two wheels, providing power to the beacons, processing the camera vision, transmission and handling of the data and algorithmically solving and mapping the maze. The development and testing of our solutions have demonstrated promising results, as evident in this report.

Some of the greatest successes in this project were being able to vertically balance the rover on two wheels while moving, and capturing and processing the video feed data in real time. However, there were also many challenges. Some of the main challenges we encountered were the long compilation time of our projects and the requirements of designing innovative solutions to novel problems, as well as integrating all subsystems into a functioning system.

In summary, we have successfully demonstrated, applied and integrated the skills we learnt in the real world and this project has been a valuable learning experience for us. Future plans include final testing of the entire system, making the UI more visually appealing as well as implement improved SMPS to interface supercapacitor with greater efficiency.

Chapter 10

References

References

- [1] “Carbon Aeronautics - YouTube,” www.youtube.com.
<https://www.youtube.com/@carbonaeronautics>
- [2] T. Technologies, “Terasic - Daughter Cards - Video Image - 8 Mega Pixel Digital Camera Package with GPIO interface,” www.terasic.com.tw. <https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&CategoryNo=68&No=1011>
- [3] “ESP32 Series Datasheet Including.” Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- [4] “OV8865,” OMNIVISION. <https://www.ovt.com/products/ov8865/> (accessed Jun. 20, 2023).
- [5] “VPI - Vision Programming Interface: Perspective Warp,” [docs.nvidia.com](https://docs.nvidia.com/vpi/algo_persp_warp.html#:text=Perspective%20Warp%20algorithm%20allows%20for).
[https://docs.nvidia.com/vpi/algo_persp_warp.html: :text=Perspective%20Warp%20algorithm%20allows%20for](https://docs.nvidia.com/vpi/algo_persp_warp.html#:text=Perspective%20Warp%20algorithm%20allows%20for) (accessed Jun. 20, 2023).
- [6] johnscience, MAZE_SOLVE_ALGOS.
https://github.com/johnscience/mazelib/blob/main/docs/MAZE_SOLVE_ALGOS.md

Chapter 11

Appendix

11.1 Robot specification

Three view diagram

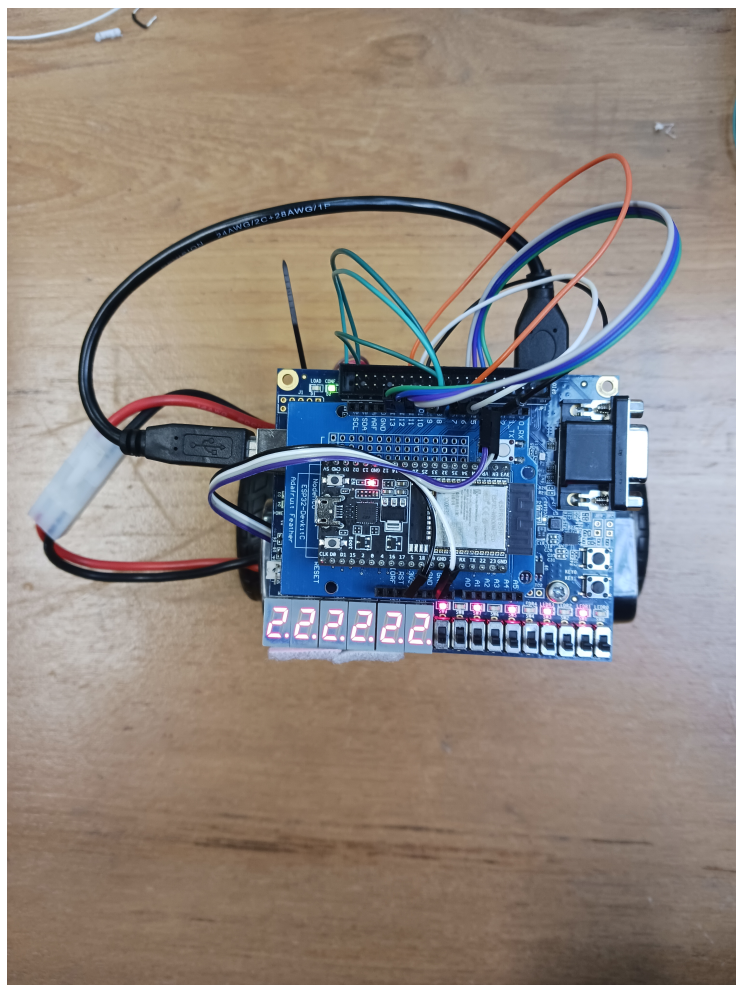


Figure 11.1: Top view

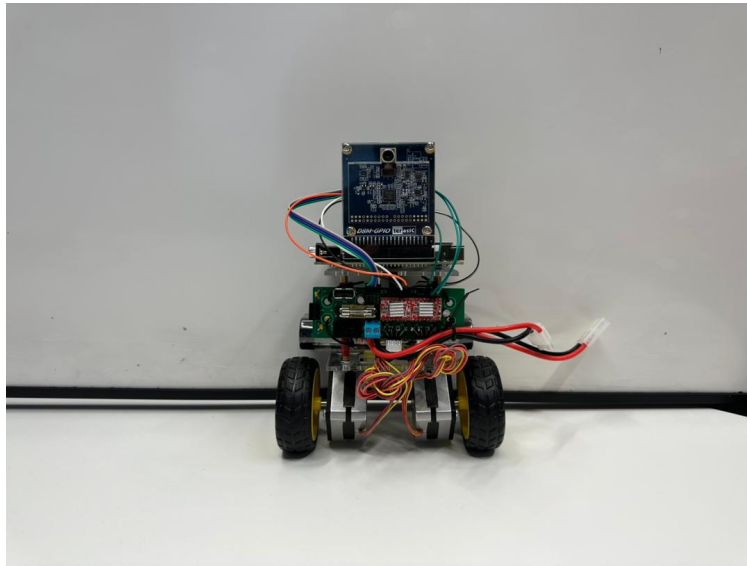


Figure 11.2: Front view

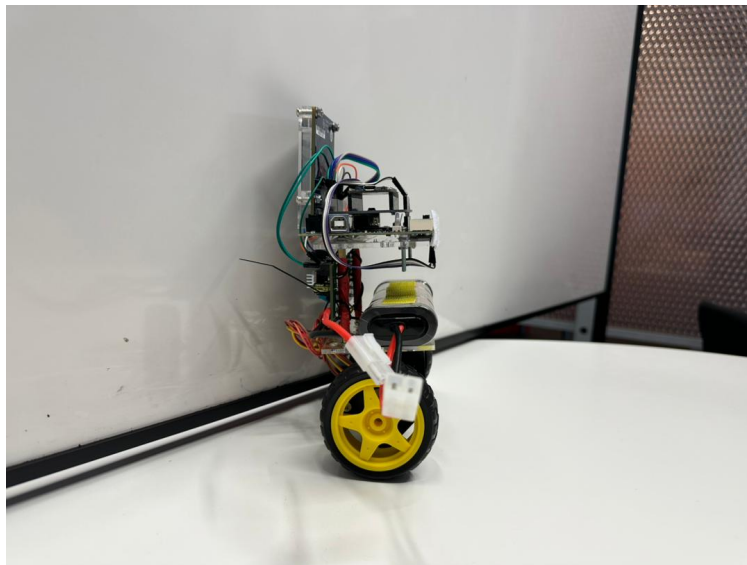


Figure 11.3: Side view

Minimum operating voltage: 6.8 V

Minimum operating current: 0.3 A

Wheel circumference: 24.1 cm

Base circumference: 44.9 cm

11.2 List of hardware components

Large chassis board $\times 1$ (Not used)
Acrylic chassis $\times 2$
NEMA17 bipolar stepper motor $\times 2$
Stepper motor PCB $\times 1$
A4988 stepper motor driver $\times 2$
7.7V, 2A battery pack $\times 1$
ESP32 WROOM dev module $\times 1$
Intel quartus lite FPGA $\times 1$
OV8865 camera module $\times 1$
Stepper motor wheels $\times 2$
Stepper motor holder $\times 2$
MPU6050 inertial measurement unit $\times 1$
ESP to FPGA connection board $\times 1$
PCB to FPGA cable $\times 1$
PCB to battery cable $\times 1$
ESP to FPGA connection board $\times 1$
M3 screw, 30mm $\times 2$
M3 screw, 10mm $\times 8$
M3 screw plastic, 10mm $\times 2$
M3 nut $\times 12$
M3 washer $\times 8$
M5 washer $\times 4$

11.3 Deduction of Positioning Algorithm

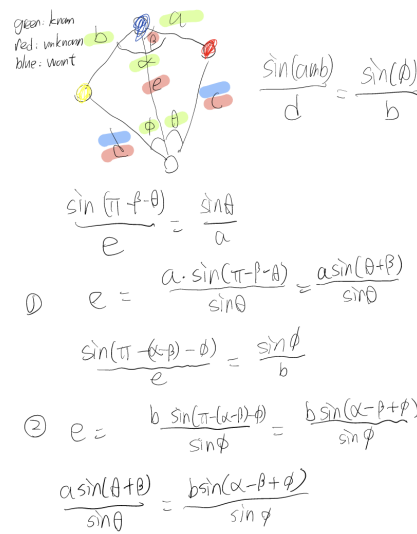


Figure 11.4: Deduction of Positioning Algorithm

$$\begin{aligned}
\frac{\sin(\beta + \theta)}{\sin(-\beta + (\alpha + \phi))} &= \frac{b \sin \theta}{a \sin \phi} \\
\frac{\sin \beta \cos \theta + \cos \beta \sin \theta}{\sin(-\beta)(\cos(\alpha + \phi)) + \cos(-\beta) \sin(\alpha + \phi)} &= \frac{b \sin \theta}{a \sin \phi} \quad \left| \frac{b \sin \theta}{a \sin \phi} = k \right. \\
\frac{\sin \beta \cos \theta + \cos \beta \sin \theta}{-\sin(\beta)(\cos(\alpha + \phi)) + \cos(\beta) \sin(\alpha + \phi)} &= k \\
\sin \beta \cos \theta + \cos \beta \sin \theta &= -k \sin \beta \cos(\alpha + \phi) + k \cos \beta \sin(\alpha + \phi) \\
\text{divide by } \cos \beta & \quad \text{can I do this?} \\
\frac{\sin \beta}{\cos \beta} \cdot \cos \theta + \sin \theta &= -k \frac{\sin \beta}{\cos \beta} \cos(\alpha + \phi) + k \sin(\alpha + \phi) \\
\tan \beta (\cos \theta + k \cos(\alpha + \phi)) &= k \sin(\alpha + \phi) - \sin \theta \\
\tan \beta &= \frac{k \sin(\alpha + \phi) - \sin \theta}{\cos(\alpha + \phi) + \cos \theta}
\end{aligned}$$

Figure 11.5: Deduction of Positioning Algorithm

$$\begin{aligned}
\tan \beta &= \frac{\frac{b \sin \theta}{a \sin \phi} \cdot \sin(\alpha + \phi) - \sin \theta}{\frac{b \sin \theta}{a \sin \phi} \cdot \cos(\alpha + \phi) + \cos \theta} \\
\text{multiply by } a \sin \phi & \\
\tan \beta &= \frac{b \sin \theta \cdot \sin(\alpha + \phi) - a \sin \phi \sin \theta}{b \sin \theta \cos(\alpha + \phi) + a \sin \phi \cos \theta}
\end{aligned}$$

Figure 11.6: Deduction of Positioning Algorithm